引言

仓颉多平台技术使仓颉开发者能够将代码集成到新的Android/iOS应用程序中，并逐步用仓颉代码替换现有应用程序中的Java/Objective-C部分。实现跨语言、跨运行时互操作的核心机制是*镜像类型*，它将一种语言的类型暴露给另一种语言。

在仓颉端，镜像类型支持通过方法重写继承Java/Objective-C类，并能实现Java接口与Objective-C协议，全程采用标准仓颉语法。在Java/Objective-C端，代表仓颉类型的镜像类型则以对应语言的原生类型形式呈现。 综上所述，该机制实现了应用程序中Java/Objective-C部分与仓颉部分的无缝衔接，包括在仓颉代码中调用目标操作系统的API。

挑战与解决方案

Java、Objective-C和仓颉都是支持继承与多态的面向对象语言。然而，它们在语义、对象模型和执行模型上的差异，使得仓颉对象无法直接用于Java或Objective-C代码中，反之亦然。

此外，三种语言均具备管理运行时环境，支持自动内存管理、线程处理、异常处理等底层功能，但实现方式各不相同。若让两个独立设计的复杂语言运行时相互感知，可能使整个系统复杂度超出人类认知范畴。

因此，仓颉与Java的互操作性是通过将二者伪装成低级语言来实现的。应用程序中的仓颉部分和Java部分通过Java本机接口（JNI）相互交互——该接口最初设计用于支持C/C++等语言开发Java本机方法。JNI虽功能丰富但属于低级API，而编写Java本机方法向来以繁琐著称。 所幸CJMP自动化消除了JNI的复杂性。

Objective-C 运行时模块 API 在 iOS 中承担类似功能。它同样专为在 Objective-C 与其他语言间创建桥接层而设计。与 JNI 类似，CJMP 负责处理繁琐的部分。

核心概念

镜像类型

以仓颉和Java作为互操作语言对为例，其中一种语言定义的*镜像类型'T'*代表另一种语言中存在的类型

T，使第一语言编写的代码能够使用该类型（可能存在某些限制）。

在两种语言中本质相同的布尔类型和数值类型自然相互映射：Cangjie类型Int32在Java中的镜像类型是int，反之亦然。对于另一种语言不自然支持的数值类型，目前尚无法实现映射，例如Cangjie类型Float16无法映射到Java的原始类型。

对于类、结构体或接口等用户定义类型，映射类型应为另一语言中最接近的等效类型。例如，唯一能以合理精度近似Cangjie结构体或元组类型的Java类型是final类。

镜像类型会暴露原始用户定义类型的成员和构造函数，使其在另一种语言中可访问*且*可用。因此，返回UInt64类型的仓颉成员函数若要映射到Java，仍需借助特殊技巧。

通常情况下，您可自动获取*Java或Cangjie类型*及其所需依赖项的镜像类型定义。针对Java类型，系统提供独立的镜像生成器；而Cangjie类型的Java镜像则由 作为cjc编译器的副产品，在常规编译过程中自动生成。

将Java类型映射至仓颉

cjc编译器会将Java镜像类型的所有使用替换为相应的粘合代码，因此仅类型名称本身及其可访问成员的名称和类型具有实际意义。因此，Java类型的镜像仅包含成员声明，不包含定义：构造函数和成员函数/属性的主体被省略，成员变量没有初始化器。基于相同原因，私有和包私有成员也被省略。

要使这种对常规仓颉语法的扩展生效，每个镜像类型都必须标记为 @JavaMirror 注解。这有助于编译器区分镜像类型声明与常规仓颉类型定义。

例如，针对以下Java类的镜像类：

```
public class Node {
    public static final int A = 0xDeadBeef;
    private int _id;
    public Node(int id) { _id = id; }
    public int id() { return _id; }
}
```

可能如下所示：

```
@JavaMirror
public open class Node {
```

```
    public static let A: Int32 public
    init(id: Int32) public open func
    id(): Int32
}
```

互操作性库中预定义了若干基础Java类型的镜像，包括java.lang.Object、java.lang.String和数组类型。

将仓颉类型映射至Java类型

与cjc编译器专门支持的Java类型映射不同，仓颉类型的映射必须在外观上完全符合常规Java类型，以便Android工具链识别。因此cjc编译器会生成Java源代码文件形式的映射，其中包含负责连接Java与仓颉世界的胶合代码。

例如，如果上一节示例中的Node类最初在Cangjie中定义如下：

```
public class Node { private let
    _id: Int public prop id: Int
    {
        get() { _id }
    }
    public init(id: Int) { this._id = id }
}
```

cjc编译器可为其生成以下Java镜像：

```
public final class Node {
    /* 部分粘合代码 */
    public Node(long id) {
        /* 粘合代码构建仓颉节点实例并关联
         * 它与正在构造的Java Node 实例相关联，即'this'。
         */
    }
    public long getId() {
        /* 胶水代码调用关联的仓颉节点实例的仓颉'id'属性获取器并返回结果。
         */
    }
    /* 更多粘合代码 */
}
```

互操作类

*互操作类*本质上是从一个或多个镜像类型派生而来的仓颉类，可在Java中使用。其所有构造函数和非继承的公共成员函数均通过共轭*包装类*暴露给Java代码，

该封装类本身不定义其他用户可调用的方法或构造函数，但其从父类型继承的任何方法均可从Java和Cangjie代码中调用。

例如编译以下Java互操作类时：

```
@JavaImpl
public class BooleanNode <: Node { private let
        flag: Bool
        public init(id: Int32, flag: Bool) {
                super.init(id)
                this.flag = flag
        }
        public func getFlag(): Bool { flag
        }
}
```

cjc编译器还将生成类似以下的Java源代码文件：

```
public class BooleanNode extends Node {
    /* 粘合代码 */
    public BooleanNode(int id, boolean flag) {
        /* 胶水代码构建 Java BooleanNode 实例并关联
         * 关联至正在构造的仓颉实例， 即'this'。
         */
    }
    public boolean getFlag() {
        /* 胶水代码调用关联的仓颉BooleanNode实例的'getFlag'成员函数并返回结果。
         */
    }
    /* 更多粘合代码 */
}
```

外部类型

镜像类型和互操作类并非完全属于其定义语言的原生类型。延续比喻，它们的地位更像是临时访客和工作签证持有者，因此本文档中统称为*外部类型*。

Java兼容类型

下列仓颉*类型被称为Java兼容*类型：

- 在Java基本类型中存在直接对应的值类型（包含Int16，但不包含UInt8）

- 外部类型
- 形式为 Option<*T*> 的类型，其中 *T* 是外部类型（参见*空值处理*部分的说明）

互操作库中包含的特殊泛型镜像类型 JArray<T> 表示 Java 数组。其类型变量 T 必须是与 Java 兼容的类型。

出于显而易见的原因，外部类型的公共成员函数的参数和返回值只能具有与相应语言兼容的类型。

互操作类的成员变量可具有任意类型。其具有Java兼容类型的公共成员变量可同时从Java和Cangjie访问。

```
@JavaImpl
class NamedNode <: Node {
    public var jName: JString          // 位于共轭包装类实例中public var cjName: String    // 位于互操作类实例中init(name: JString) {
        jName = name
        cjName = name.toCangjieString()
    }
}
```

互操作场景

Java与仓颉之间主要存在两种交互场景：

- 一种场景是将仓颉输入法功能暴露给Java，使Java代码能够使用仓颉API、库及应用组件。
- 在另一种场景中，Java类和接口被暴露给仓颉，使仓颉代码能够访问Android API、第三方Java库以及应用程序中保留在Java中的组件。

上述场景并非互斥，单个应用程序亦可同时采用两种模式。此外，两种场景均支持类继承与接口实现（Java类可继承仓颉类，反之亦然）。这使得回调机制无缝衔接，因此两种模式均非单向操作。但每种场景具有各自的功能集、限制条件及工具支持等特性，故需分别阐述。

在Java中使用仓颉

要在 Java 中访问仓颉库、API 或其他代码片段，需为需要暴露的仓颉类型生成 Java 镜像类型。cjc 编译器可完成此操作，具体方法详见《仓颉镜像生成参考》。

注：仓颉语言的类型系统比其Java对应版本更为丰富且灵活。两种语言共享的特性也存在若干主要差异，其中最显著的是泛型支持。因此某些仓颉语言特性无法在Java中实现，而其他特性则难以用自然紧凑的方式表达。具体支持特性及相关限制请参阅《仓颉到Java映射表》。

示例：

假设您希望向Java暴露一个定义如下、名为Vector的仓颉结构体：

```
package cj

public struct Vector { private var x:
    Int32 = 0 private var y: Int32 = 0

    public init(x: Int32, y: Int32) { this.x = x
        this.y = y
    }

    public func add(v: Vector): Vector { Vector(x + v.x,
        y + v.y)
    }
}
```

在编译 Vector.cj 时，需向 cjc 编译器添加两个额外选项：--experimental --enable-interop-cjmapping=Java。编译器将生成一个额外的文件 Vector.java，内容类似如下：

```
package cj;

public final class Vector {
    // 粘合代码

    public Vector(int x, int y) { /* 粘合代码 */ }

    public Vector add(Vector v) { /* 粘合代码 */ }

    // 更多粘合代码
}
```

现在，您可以在Android项目的src/main目录下添加Vector.java文件，并在Java代码中像使用普通Java类一样使用Vector类：定义该类型的变量、创建实例、传递实例（包括调用Vector.add等方法）、应用类型比较和类型转换运算符等。

限制暴露范围

选项 --enable-interop-cjmapping 会指示 cjc 编译器为其编译的所有公共仓颉类型生成镜像类型，并暴露这些类型*的所有*公共成员函数和构造函数。然而，这种完全暴露往往并不理想，因为您在 Java 中需要使用的功能可能仅占仓颉库或框架所提供功能的一小部分。

您可通过向cjc传递专用配置文件的路径名来精确控制仓颉类及其成员的可见性，使用选项 --import-interop-cj-package-config-path 向 cjc 传递专用配置文件的路径名，精确控制仓颉字包及其成员的暴露范围。该文件为纯文本文件，内容必须符合 TOML 语法规范。

在文件的[default]部分，可为所有仓颉包指定默认API暴露策略，随后可在对应的[[package]]条目中针对特定仓颉包进行修改。例如：

```
[default]
APIStrategy="None"              # 默认不暴露任何接口

[[packages]]
name="com.example.pkg1"         # 从此特定包开始，
API策略="完整"                  # 暴露所有内容
```

此外，对于每个包，您可以使用included_apis或excluded_apis属性分别暴露或隐藏特定类型和/或成员：

```
[[packages]]
name="com.example.pkg2"                 # 从此特定包中，
included_apis = [ "Vector",             # 仅包含类型 'Vector'
                  "Vector.add"          # 及其成员函数 'add'
                ]                       # 暴露

[[packages]]
name="com.example.pkg3"                 # 从此特定包开始，
API策略="完整"                          # 所有内容
excluded_apis = [ "TopSecret",          # 但类型 'TopSecret' 和
                  "Auth.getPwd"         # 成员函数 'Auth.getPwd'
                ]                       # 暴露
```

更多详情请参阅仓颉镜像生成参考。

通用仓颉类型映射

许多重要的仓颉类型都是泛型的。遗憾的是，仓颉与Java泛型之间存在某些根本性差异，导致前者无法直接映射到后者。作为替代方案，仓颉SDK支持映射类型的*单态化*：它会为给定参数化仓颉类型的每组有效类型参数，生成独立的非泛型Java映射类型。

例如，若存在泛型仓颉类 p.Pair<T,U>：package p
public class Pair<T,U> { . . . }

若需在Java代码中操作Pair<Int, String>和Pair<Foo, Bar>的实例，需在配置文件的[[package]]条目下为p添加generic_object_configuration属性，内容如下：

```
#     -    -    -
[[package]]
name = "p" generic_object_configuration   = [
      { name = "Pair", type_arguments = [
                "整数, 字符串",
                "Foo, Bar"
        ] },
#           -    -    -
]
```

cjc 随后将生成两个独立的 Java 类：

```
public final class GIntString { . . . }
```

```
public final class GFooBar { . . . }
```

代表类型 Pair<T,U> 的特定*实例化形式*。

注意：当前版本仅支持布尔型和数值型作为单态泛型类型的参数，因此无法实际暴露Pair<Foo, Bar>。

更多细节请参阅泛型实例化。

仓颉到Java映射

当前版本的cjc编译器采用本章所述的仓颉→Java映射。适用cjc命令行选项请参阅仓颉镜像生成参考（）。

通用注意事项

仓颉类型系统比Java语言的类型系统灵活得多。许多仓颉类型及其特性在Java中没有直接对应或甚至接近的等价物，因此某些仓颉类型在当前版本中支持有限，有些则完全不被支持。

因此，若公共成员、函数/构造函数参数或函数返回值的类型不受支持，则对应成员、函数或构造函数无法被镜像。编译器会将此类用法标记为错误。但若该类型出现在不受*支持的上下文中*（例如作为公共成员变量的类型），则对应实体仅不被镜像，且不会触发 错误。

类型别名

类型别名声明*不会*被镜像，任何类型别名使用都会被视为已替换为别名类型。

名称

仓颉包、函数、类型及类型成员的原始名称目前得以保留，这意味着其中部分名称可能与Java关键字（如"int"或final）发生冲突，或包含Java名称中不允许的字符（）。

布尔与数值类型

在Java基本类型中存在对应的仓颉布尔型和数值型将映射至相应基本类型；未对应的类型不被支持，包括IntNative和UIntNative类型：

仓颉类型	Java类型
Bool	boolean
Int8	byte
Int16	短整型
Int32	整型
Int64	longIntNative
	不支持 UInt8
	不支持
UInt16	char
UInt32	不支持
UInt64	不支持
UIntNative	不支持 Float16 不支持
Float32	float

仓颉输入法	Java 类型
Float64	double

有关不支持类型的处理信息，请参阅一般注意事项。

符文

Rune 类型不受支持。

有关未支持类型的处理信息，请参阅一般注意事项。

字符串

字符串类型尚未支持。

有关未支持类型的处理信息，请参阅一般注意事项。

数组

数组类型（Array<T>实例化）尚未支持。

有关未支持的 类型的处理信息，请参阅一般注意事项。

元组

仓颉元组类型会被映射到最终的Java类定义中。这些定义由cjc编译器自动生成，包含用于在Java和仓颉之间传递控制和数据的粘合代码。请勿手动修改这些定义。

```
package tuples

import interoplib.interop

public class SimpleTupleExample {
    public static func z(): (Int, Int) { (0, 0)
    }
}
```

```
// java-gen/SimpleTupleExample.java
package tuples;
// 粘合代码
```

```
public class SimpleTupleExample {
    // 粘合代码
    public static native TupleOfInt64Int64 z();
    // 粘合代码
}
// java-gen/TupleOfInt64Int64.java
package tuples;
// 粘合代码
final public class TupleOfInt64Int64 {
    // 粘合代码
    public TupleOfInt64Int64(long item0, long item1) {
        // 粘合代码：构造一个仓颉字符集（整型，整型）元组并将其关联
        // 关联到正在构造的Java对象，即'this'。
    }

    public long item0() {
        // 从仓颉字典中获取第0个元素的粘合代码 （Int，Int）
        // 元组关联'this'并返回。
    }

    public long item1() {
        // 粘合代码： 从与'this'关联的仓颉(Int，Int)元组中获取第1个元素
        // 元组关联'this'并返回。
    }
    // 粘合代码
}
```

元组类型 (*T0*，*T1*，... *Tn*) 被映射到最终的 Java 类，具有以下属性：

- 若任何 *Ti* 是类型别名，则元组类型将被镜像处理，视该 Ti_ 为别名类型。例如，(Int, Int) 将被视为实际类型 (Int64, Int64) 进行处理。
- 镜像类的名称为 TupleOf *T0T1* ...*Tn*（即元组元素的*仓颉*类型名称拼接后，前缀 TupleOf）。 例如，由于上述别名替换机制，仓颉元组类型 (Int, Int) 的 Java 镜像类名称将变为 TupleOfInt64Int64（如前例所示）。

  注意：尚未支持不具备独立简单名称（非别名）的类型作为元组元素类型。有关未支持类型的处理方式，请参阅通用注意事项。
- 镜像类拥有一个带参数的公共构造函数，参数类型*为T0'*、 *T1'*、 ...、*Tn'*，其中每个*Ti'*对应Java中*Ti*的镜像类型。该构造函数创建一个镜像类型的仓颉元组，其元素值

并将其关联至正在构造的Java对象（即this）。详见上文示例。

- 对于每个元素类型 *Ti*，镜像类都提供了一个无参实例方法 public *Ti′ itemi* ()，该方法返回构造时接收者关联的仓颉元组中第 i 个元素的镜像。*Ti′* 仍是 *Ti* 的 Java 镜像类型。

当前实现存在以下限制：

- 元组元素类型的支持仅限于数值型和布尔型。有关未支持类型的处理方式，请参阅通用注意事项。
- 编译器不会在遇到其他镜像实体定义中的元组类型使用时自动生成镜像。所有此类元组类型必须在配置文件中显式列出：

  ```
  #      -      -     -
  [[package]]
  name = "tuples"
  tuple_configuration = [ "(Int, Int)" ]
  #      -      -     -
  ```

  编译器在遇到必须镜像但未在相应仓颉包的元组配置数组中列出的元组类型时会报告错误。

- 仓颉元组类型具有*结构性*，即当两个元组类型具有相同数量的元素且各自的元素类型相同时，它们是兼容的。 此规则同样适用于元组类型的别名。然而，映射仓颉元组的Java类类型之间不存在子类关系，因此不可兼容。若两个或多个暴露给Java的仓颉包中交叉使用本质相同的元组类型，可能引发兼容性问题。
- 仓颉元组类型属于值类型，而其对应的Java类型则是引用类型。由此产生两方面影响：
  - 多个Java变量可能引用同一个Cangjie元组。由于元组不可变，这本身不构成问题。
  - 比较运算符==和!=具有完全不同的语义。在Java中，它们测试镜像类实例的引用相等性，而在Cangjie中则按元素逐项比较。实际上，若两个结构相同的Cangjie元组中存在不支持相应运算符的元素类型，则可能无法比较。

    此外，在当前实现中，元组类型的镜像类并未实现equals方法（更准确地说，生成的覆盖方法会抛出 UnsupportedOperationException），

因此在Java端无法简洁地比较两个同类型元组。

范围类型

目前尚未支持范围类型。

有关未支持类型的处理方式，请参阅通用注意事项。

特殊类型

类型 Unit 仅作为函数返回类型支持，映射到 Java 的 void。

类型 Nothing 无法支持。类型 Any 目前尚未支

持。

有关未支持类型的处理信息，请参阅一般注意事项。

函数类型

仓颉函数类型映射为Java函数接口声明：

```
package cjworld

import interoplib.interop

public class IntFuncBox {    private let
        _f: (Int) -> Int    {        public
        init(f: (Int) -> Int) {            _f = f
        }    }    public func unbox(): (Int) -
        > Int {        _f = f    }
        public init(f: (Int) -> Int) { _f = f }public func
        unbox(): (Int) -> Int { _f }
}
```

```
package cjworld;
// 粘合代码

@FunctionalInterface
public interface Int64ToInt64
{
// 粘合代码

    public  long call(long p1);

// 粘合代码
}
```

```
package cjworld;
// 粘合代码

public class IntFuncBox {
// 粘合代码

    public IntFuncBox(Int64ToInt64 f) {
        // 粘合代码创建仓颉类 IntFuncBox 的实例
        // 并将该实例关联至正在构造的对象， 即'this'。
    }

    public Int64ToInt64 unbox() {
        // 胶水代码调用关联'this'的Cangjie类的unbox()实例成员函数
        // 并将其封装为
        // 实现 Int64ToInt64 的辅助类实例。
    }

//    粘合代码
}
```

函数类型 (*T1*，*T2*，… *Tn*) -> U 将映射为具有以下特性的 Java 函数接口：

- 若存在类型别名 Ti，则函数类型映射时将视该 *Ti* 为别名类型。例如 (Int) -> Int 将被处理为实际的 (Int64) -> Int64。
- 若存在 *Ti* 和/或 *U* 是类型变量，则针对每个泛型类型或成员的单态化结果分别映射函数类型。详见泛型实例化中的示例。
- 镜像接口的名称为 *T1T2 …Tn ToU*（即参数的*仓颉*类型名称拼接后，接 To 字符，再接结果类型名称）。例如，仓颉函数类型 (Int, Bool) -> Int 的 Java 镜像接口名称将因前述别名替换而变为 Int64BoolToInt64。

  注意：尚未支持作为函数元素类型的类型若无独立简单名称（非别名），请参阅通用注意事项了解未支持类型的处理方式。

- 镜像接口提供单个公共实例方法：

  U' call(*T1' p1* ，*T2' p2* ，… *Tn' pn*) { … }

  其中每个 *Ti'* 对应 *Ti* 的 Java 镜像类型，*U'* 对应 *U* 的 Java 镜像类型（若 *U* 为 Unit 则为 void）。参数将自动命名为 p1, p2, ..., *pn*。

- 镜像接口类型可用于双向传递函数和lambda表达式：
  - 从仓颉传递至Java的该类型值（例如由Java调用的镜像仓颉成员函数返回值），将关联至*仓颉*函数类型的值（例如lambda表达式）。 调用方法 call() 可执行该仓颉函数，将参数作为镜像仓颉类型 *T1*、 *T2*、 ... *Tn* 的值传递，并返回该仓颉函数返回的类型 *U* 的镜像值（若函数返回 Unit 则返回空值）。
  - 镜像类型在所有方面都是标准的Java函数接口类型，因此不仅Java类可以实现它，匹配类型的lambda表达式也与其兼容。 当此类Java类的实例或兼容的lambda表达式传递给仓颉时，会合成一个镜像的仓颉函数类型值(*T1*, *T2*,...*Tn*) -> U，并与该Java lambda或类实例关联。最终调用该函数的仓颉代码将调用Java lambda或方法。

当前实现存在以下限制：

- 函数参数与返回值类型仅支持数值类型、Bool及Unit。未支持类型的处理方式详见通用注意事项。
- 编译器*不会*在遇到其他镜像实体定义中的函数类型使用时自动镜像该类型。所有此类类型必须在配置文件中显式列出：

```
#      -      -     -
[[package]]
name = "lambdas" lambda_patterns =
[
      { 签名 = "(整数) -> 整数" },
      { 签名 = "(Float64) -> Bool" },
#     -      -     -
]
```

  编译器在遇到必须镜像但未在相应仓颉包的lambda_patterns数组中列出的函数类型时会报告错误。

- 仓颉函数类型*具有结构性*，即当两个函数类型具有相同参数个数、相同参数类型及相同返回类型时，它们即兼容。此规则同样适用于函数类型的别名。然而，若两个Java函数接口类型在不同仓颉包中映射了本质相同的仓颉函数类型，则两者不兼容。当此类

函数类型在两个或多个暴露给Java的仓颉包之间交叉使用时可能引发问题。

- 仓颉函数类型不可比较，而其Java映射类型可比较。这不应构成问题。

结构体类型

公共仓颉结构体类型定义被映射到最终的Java类定义中。这些定义由cjc编译器自动生成，包含用于在Java与仓颉之间传递控制流和数据的粘合代码。请勿手动修改这些定义。

```
package cj

import interoplib.interop.*

public struct Vector { private var x:
    Int32 = 0 private var y: Int32 = 0

    public init(x: Int32, y: Int32) { this.x = x
        this.y = y
    }

    public func add(v: Vector): Vector { let res =
        Vector(x + v.x, y + v.y)
        print("cj: (${x}, ${y}) + (${v.x}, ${v.y}) = (${res.x}, ${res.y})\n", flush: true) return res
    }

    public static func dump(v: Vector): Unit {
        print("cj: 来自 cj.Vector 静态函数的问候 (${v.x}, ${v.y})\n", flush: true)
    }
}
```

```
package cj;

// 粘合代码导入

final public class Vector {
    // 粘合代码

    public Vector(int x, int y) {
        // 粘合代码，构造仓颉向量结构体的实例
        // 并将其关联到 Java 对象 'this'
```

```
    }

    public Vector add(Vector v) {
        // 粘合代码，用于检索关联的仓颉向量结构体
        // 关联`this`和`v`的Cangjie向量实例，调用Cangjie-this的add()实例函数
        // 实例函数，将 Cangjie-v 作为参数传递
        // 作为参数传递，最后创建新的Java向量实例
        // 并将该实例关联至add()调用结果。
    }

    public static void dump(Vector v) {
        // 调用结构体 Vector 静态成员函数 dump() 的胶水代码
        // Cangjie struct Vector的静态成员函数dump()，并向其传递关联实例
        // 'v'/
    }

    // 更多粘合代码
}
```

仅镜像公共结构体成员与通用构造函数。

成员函数将映射为方法，其中参数类型和返回值类型将替换为对应的镜像类型。返回 Unit 的成员函数将映射为 void 方法。修饰符 public 和 static 将被保留。

常见构造函数会被镜像为构造函数，其中参数类型会被替换为相应的镜像类型。*这包括可能隐式声明的默认构造函数*。修饰符 public 将被保留。

当前版本对可镜像的结构体类型施加了诸多严格限制，甚至可以说结构体类型必须专门为暴露给Java而设计：

- 成员变量不会被镜像，且不提供访问途径。此限制将在未来版本中解除。当前可通过添加获取函数作为临时解决方案。
- 可变实例成员函数的镜像功能尚在开发中。虽然已实现镜像，但在某些情况下生成的粘合代码存在错误。
- 不支持镜像实现除 Any 之外接口的结构体。尝试镜像显式实现接口的结构体将导致编译时错误。
- 泛型结构体的镜像功能通过单态化实现。详情参见《泛型》章节。
- 不支持包含公共成员属性、成员运算符函数或初始构造函数的结构体镜像。尝试

镜像包含上述任一类实体的结构体将导致编译时错误。

- 若尝试镜像结构体成员函数或构造函数时使用了不受支持的参数类型，或成员函数具有不受支持的返回类型，将导致编译时错误。

类与接口

仓颉类和接口定义分别映射为 Java 类和接口定义。这些定义包含自动生成的粘合代码，不得进行任何修改。

```
package cj

import interoplib.interop.*

public interface Valuable { public func
    value(): Int
}

public open class 单例 <: 珍贵 { private let _v: Int
    public init (v: Int) { _v = v } public func
    value(): Int { _v }
}

public class Zero <: Singleton { public
    init() { super(0) }
}
```

```
// Valuable.java
package cj;

// 粘合代码导入

public interface Valuable {
    public long value();
}

// 粘合代码

// Singleton.java
package cj;

// 粘合代码导入

public class 单例实现类 implements 价值类 {
```

```
    // 粘合代码

    public Singleton(long v) {
        // 粘合代码，用于构造仓颉单例类的实例
        // 并将其关联到Java对象'this'
    }

    public long value() {
        // 粘合代码：获取关联的仓颉单例类实例
        // 调用其成员函数value()并返回结果
    }

    // 粘合代码
}
package cj;

// 粘合代码导入

public final class Zero extends Singleton {
    // 粘合代码

    public Zero() {
        // 粘合代码，用于构造仓颉零类的实例
        // 并将其关联到Java对象'this'
    }

    // 粘合代码
}
```

当前，即使镜像类型及其成员的标识符与byte等Java关键字冲突，这些标识符仍会被保留。

从外部视角看，镜像类型在所有方面都是标准的Java类和接口：除final类外均可被继承/实现，其方法可在子类型中通过常规Java方式重载，非final方法可被覆盖，实例类型可通过instanceof进行比较，等等。

成员函数被映射为方法，其中参数类型和返回值类型被替换为相应的镜像类型。返回Unit的成员函数映射为void方法。映射非公开成员函数的方法会被添加final修饰符。public和protected修饰符（详见下文）得以保留。 类成员函数的 static 修饰符将被保留。参数名称保持不变。**警告：**静态接口成员函数目前会被映射为实例方法，但对于实现此类接口的仓颉类，其映射代码生成存在错误。

成员属性本身不被映射，但其获取器和设置器会被映射为方法。 这些方法名称分别以“get”或“set”开头，后接属性名称（首字母大写以保持驼峰式命名）。属性获取方法镜像为无参数方法，返回值类型与属性类型一致；属性设置方法镜像为void类型方法，带有一个与镜像类型相同的参数。修饰符public、protected（详见下文）和static均被保留。

构造函数会被镜像为构造函数，其中参数类型会被替换为相应的镜像类型。*这包括可能隐式声明的默认构造函数。*修饰符 public 将被保留。参数名称将被保留。

通常仅以下实体需进行镜像处理：

- 上述列举类型的公共构造函数及成员
- 开放类中的受保护公开成员（包括受保护的抽象成员函数），因其可在Java端被重写。

成员变量和**成员运算符函数**目前不进行镜像处理。

由于仓颉不支持类型嵌套（与Java不同），镜像类型永远不会包含嵌套类型（除胶合代码使用的私有成员类型外）。

当前版本对可镜像的类和接口类型施加了诸多严格限制，甚至可以说此类类型必须专门为暴露给仓颉而设计：

- 抽象类不予镜像。
- 成员变量不予镜像，且不提供访问途径。此限制将在未来版本中解除。过渡期间，可通过手动为仓颉类添加获取/设置函数作为临时解决方案。
- 泛型类的镜像通过单态化实现。详见泛型说明。
- 成员运算符函数的镜像功能尚未支持。若尝试镜像包含此类实体的类或接口，将导致编译时错误。
- 若尝试镜像类成员函数或构造函数时使用了不受支持的参数类型，或镜像成员函数时使用了不受支持的返回类型，将导致编译时错误。

枚举

仓颉枚举会被镜像为一个没有公共构造函数的最终Java类。

```
public enum TimeUnit {
    | 年(Int64)
    | 月(Int64)
    | 年
    | 月
}
```

```
public final class TimeUnit {
    /* 粘合代码 */

    public static TimeUnit Year(long p1) {
        // 胶水代码创建仓颉字符集的Year(p1)枚举并关联
        // 与新创建的Java TimeUnit 实例关联
    }

    public static TimeUnit Month(long p1) {
        // 胶水代码创建 Cangjie Month(p1) 枚举并将其关联
        // 与新创建的Java TimeUnit 实例关联
    }

    public static TimeUnit 年 =
        // 胶水代码创建仓颉字母Year枚举并关联
        // 与新创建的Java TimeUnit 实例关联

    public static TimeUnit Month =
        // 胶水代码创建仓颉年枚举并关联
        // 与新创建的Java TimeUnit 实例关联

    /* 更多粘合代码 */
}
```

无参数的枚举构造函数会被映射为静态变量，这些变量通过与仓颉枚举相应构造函数关联的类实例进行初始化。

带参数的枚举构造函数会被映射为同名静态工厂方法，参数类型与数量保持一致；参数名称将被合成（p1, p2, ...）。

成员函数映射为方法，其中参数类型和返回值类型替换为对应的镜像类型。返回Unit的成员函数映射为void方法。修饰符public和static得以保留。

成员属性本身不映射为任何内容，但其获取器（枚举无法拥有可变属性）会被映射为方法。这些方法名称以“get”开头，后接属性名称（首字母大写以保持驼峰命名法）。映射属性的方法

获取器不带参数，返回值类型与属性类型一致。修饰符 public 和 static 得以保留。

当前版本对可镜像枚举施加了若干限制：

- 不支持镜像实现除 Any 之外接口的枚举。尝试镜像显式实现接口的枚举将导致编译时错误。
- 泛型枚举的镜像通过单态化实现，详见泛型章节。
- 不支持包含成员运算符函数的枚举镜像。尝试镜像此类枚举将导致编译时错误。
- 若尝试镜像具有不支持参数类型的枚举成员函数或构造函数，或具有不支持返回类型的成员函数，将导致编译时错误。

枚举扩展（extend）中定义的成员目前会被镜像为枚举本身的成员。

递归定义的枚举受支持：

```
public enum Peano {
    | Z
    | S(Peano)

    public func toInt(): Int { match
        (this) {
            case Z => 0
            case S(x) => 1 + x.toInt()
        }
    }
}
```

泛型

Java与仓颉泛型的本质差异决定了后者无法直接映射至前者。然而，仓颉参数化类型的具体实例化形式（如G<Int64>）属于具体类型，因此可映射为非泛型的Java类型。此类类型*亦称为单态化泛型*。

请通过cjc编译器的配置文件指定需映射的类型实例化形式。详情参见《泛型实例化说明》。

示例：

编译以下仓颉类定义时：

```
public class G<T> {
    private let t: T
    public init(t: T) { this.t = t } public func
    get(): T { t }
}
```

并在配置文件相应部分添加以下内容：

```
    - - -
通用对象配置                                = [
    { name = "G", type_arguments = ["Bool", "Int"] },
]
    - - -
```

编译器将为 G<Bool> 和 G<Int> 生成 Java 镜像类，分别命名为 GBool 和 GInt。

完整说明请参阅《仓颉镜像生成参考》章节。

null 处理

由于仓颉没有空值类型，将空值作为参数传递给镜像仓颉方法会导致胶合代码中出现运行时异常：

```
public class Node {
    private let next: ?Nodepublic
    init() {
        next = None
    }
    public init(next: Node) {                    /* 纯仓颉输入法 */
        this.next = Some(next)
    }
}
public final class Node {   Node() { /*
    粘合代码 */ }
    Node(Node next) { /* 粘合代码 */ }
}
  - - -
    节点列表 = new 节点(null);              /* NullPointerException */
```

相反地，目前尚无法从Java兼容类型的公共成员函数中返回null。

注意：当前版本支持Option<T>的自动(解)封装，其中null映射为None，但仅限于映射至苍楞的Java引用类型（），反向映射不支持。详见null处理说明。

仓颉映射生成参考

在仓颉SDK中，不存在独立的镜像生成器用于将仓颉字型暴露给Java和Objective-C代码。cjc编译器本身在编译仓颉字型的源代码时，即可为其生成对应的Java/Objective-C镜像字型定义。

命令行语法

以下附加cjc选项可在编译过程中启用并控制仓颉类型的镜像生成：

--experimental *(必选)*

当前版本必须指定此选项，因跨平台功能尚在开发中，但未来将不再需要。

--enable-interop-cjmapping=Java *或*
--enable-interop-cjmapping=ObjC *(必选)*

分别启用生成Cangjie字符集的镜像类型定义，格式为Java或Objective-C源代码文件。

--output-interop-cjmapping-dir *路径名* *(可选)*

*路径名*必须指向一个目录，cjc编译器将在该目录中放置其生成的包含镜像类型定义的Java或Objective-C源代码文件。若*路径名*未指向现有文件系统位置，cjc将尝试在此处创建目录。若*路径名*指向非目录结构，cjc将终止并输出错误信息。

*路径名的*默认值为./java-gen或./objc-gen，具体取决于--enable-interop-cjmapping参数分别设置为“Java”或“ObjC”。

--import-interop-cj-package-config-path *路径名* *（可选）*

*路径*名必须指向定义哪些仓颉类型作为镜像类型暴露给 Java 或 Objective-C 的配置文件。详情请参阅仓颉镜像生成配置。

仓颉镜像生成配置

仓颉镜像生成配置文件采用TOML语法的纯文本格式，其中指定：

- 非通用仓颉类型需映射
- 泛型仓颉类型的具体实例需镜像

默认值

[default] 表定义了以下包的默认设置：要么完全未提供包特定设置，要么这些设置不会覆盖默认值。

属性：

API策略 *（可选）*

字符串，用于定义包中定义的所有公共实体是否默认进行镜像。有效值：

- "Full" *（默认）* - 所有公共实体默认进行镜像
- "None" - 默认不镜像任何公共实体

通用类型策略 *（可选）*

定义泛型公共实体是否默认镜像的字符串。有效值：

- "部分" - 在generic_object_configuration中显式列出的实例化应进行镜像
- "None" *（默认）* - 默认情况下不镜像任何泛型公共实体

包

[[packages]]数组中的每项条目指定一个独立的源仓库包名称，并可选包含：

- 一个过滤器，用于定义该包中哪些类型应包含在镜像集中，或从镜像集中排除
- API镜像策略（若非默认值）
- 通用类型镜像策略（若非默认值）
- 需镜像的特定通用类型实例化

属性：

名称 *（必填）*

包含*源*仓库名称的字符串，本[[packages]]数组条目中的设置将应用于该仓库。

示例：

name = "com.example.VectorMath"API策略

（可选）

一个字符串，用于定义是否默认镜像指定包中的所有公共实体。有效值：

- "Full" - 所有公共实体默认进行镜像
- "None" - 默认不镜像任何公共实体

若此属性缺失，则采用默认值。

included_apis *(可选)*

字符串数组，每个元素包含需镜像的公共实体名称。若数组包含类型成员的限定名称，则该类型本身也将被镜像。在给定的 [[packages]] 数组条目中，included_apis 或 excluded_apis 仅可存在其中一项，不可同时存在。

示例：

```
included_apis = [
    "Vector.product", // Vector 亦已暴露
    "HiddenV"
]
```

excluded_apis *(可选)*

一个字符串数组，每个字符串包含给定包中定义的公共仓颉实体名称，这些实体*不应*被镜像。在给定的[[packages]]数组条目中，可以包含included_apis或excluded_apis，但不能同时包含两者。

示例：

```
excluded_apis = [
    "Vector.product", // 即使暴露了Vector，product也不会被镜像
    "HiddenV"
]
```

通用类型策略 *(可选)*

定义本包中定义的泛型公共实体是否默认进行镜像的字符串。有效值：

- "Partial" - 仅镜像 generic_object_configuration 中显式列出的实例化内容
- "None" - 默认不镜像任何泛型公共实体若该属性缺失，则采用默认值。

lambda模式 *(可选)*

一个（可能为空）的表数组，每个表描述一种函数类型，该类型被一个或多个暴露实体使用，因此必须进行镜像。

数组的每个条目都具有以下必备属性：

签名 *(必填)*

包含仓颉函数类型的字符串，例如“(Int) -> Int”。

类型

字符串数组，每个字符串包含名称属性中指定的泛型类型的有效类型参数。

示例：

```
tuple_configuration = [ { signature = "(Int) -> Int" },
                        { signature = "(Float64, Float64) -> Float64" }
                      ]
```

详情请参阅函数类型。

元组配置 *（可选）*

字符串列表，每个字符串包含一个元组类型，该类型被一个或多个暴露实体使用，因此必须进行镜像处理。

示例：

```
tuple_configuration = [ "(Int, Bool)",
                        "(Float64, Float64, Float64)"]
```

详情请参阅元组。

generic_object_configuration *（可选）*

定义通用实体具体实例化的表列表，这些实例化将被 镜像。详情参见通用实例化。

泛型实例化 [[packages]]数组条目的generic_object_configuration属性定义了将镜像哪些特定的泛型仓颉实体实例化。

该属性的值是一个（可能为空）的表数组。数组中的每个条目都必须包含以下两个属性：

name

包含当前包中定义的公共通用仓颉类型的名称的字符串。

type_arguments

字符串数组，每个元素包含适用于名称属性所指定泛型类型的有效类型参数。

注意：当前版本仅支持数值类型、Bool 及 Unit 作为单态化的类型参数。若在类型数组元素中指定其他类型，将导致编译时错误。

> 简言之，“有效”意味着该类型/函数可使用尖括号<>内的给定字符串进行实例化。
>
> 考虑以下仓颉类：

```
public class G<T> {
    public func f(t: T): Unit {}public func
    f(b: Bool): Unit {}
}
```

G<Bool> 无法实例化，因此泛型对象配置

```
                                                = [
    { name = "G", types = ["Bool"] }
]
```

将触发编译器错误。

示例：

1. 以下是仓颉定义示例public class G<T> { . . .

```
}
public func f<T>(): Unit { . . . } public
struct S<T,U> { . . . }
```

属性

```
generic_object_configuration                = [
    { name = "G", type_arguments = ["Int"] },
    { name = "f", type_arguments = ["整数", "布尔值"] },
    { name = "S", type_arguments = ["整数, 布尔值"] }
]
```

指示cjc编译器为以下实例生成镜像：

G<整型> f<整

型> f<布尔型>

S<整型, 布尔型

>

并将其命名为 GInt、fInt 等。

2. 给定以下仓颉源代码文件：

```
package cjworld

import interoplib.interop.*

    public class Composer<T,U,V> {
                public static func compose(f: (T) -> U, g: (U) -> V): (T) -> V {
            { t: T => g(f(t)) }
    }
}
```

以及以下配置文件：

```
[[package]]
name = "cjworld"
APIStrategy = "Full"
通用类型策略 = "部分"

lambda_patterns = [
    { signature = "(Int) -> Float64" },
    { 签名 = "(Float64) -> Bool" }
]

generic_object_configuration = [
        { name = "Composer", type_arguments = ["Int64, Float64, Bool"] },
        { name = "Composer<Int64, Float64, Bool>", symbols = ["compose"] },
]
```

编译器将生成以下非参数化镜像类型：

- 功能接口 Int64ToFloat64：

```
// - - -
@FunctionalInterface
public interface Int64ToFloat64
{
    public double call(long p1);
    // - - -
}
```

- 功能接口 Float64ToBool

```
// - - -
@FunctionalInterface
public interface Float64ToBool
{
    public boolean call(double p1);
    // - - -
}
```

- 类 ComposerInt64Float64Bool：

```
// - - -
public class GenClass3Int64Float64Bool {
    // - - -
    public static Int64ToBool 组合(Int64ToFloat64 f, Float64ToBool g) {
        // - - -
    }
    // - - -
}
```

在仓颉编码中使用Java

在首个互操作场景中（详见《Java 中使用仓颉》），仓颉类型被映射为行为完全符合常规 Java 类与接口的 Java 类型：非 final 类可被继承，接口可被实现，实例可自由创建与传递等。

相比之下，由于Android运行时环境（JVM）的技术限制，仓颉中对Java类型的任何使用都必须完全局限于互操作类的构造函数和成员函数内部。此外，Java类与接口的镜像类型声明需通过独立工具生成，而非由cjc编译器完成。

因此实现第二种场景的过程更为复杂，具体步骤如下：

1. 基于*Java*类与方法设计互操作层。开发者 → 互操作层设计（Java*伪代码*）
2. 为互操作性层设计中使用的任何现有Java类和接口生成镜像声明。

   .class → 镜像生成器 → .cj（镜像文件）
3. *使用仓颉编写*构成互操作层的类。按需使用镜像生成的Java类型：创建实例、调用方法等。

   互操作层设计 + .cj（镜像）→ 开发者 → .cj（互操作层）
4. 将互操作类与镜像类型声明一同编译为
   cjc。cjc将生成：

   - 所有镜像类型使用场景所需的粘合代码。
   - 互操作层Java部分的实际Java源代码。

   .cj（镜像层 + 互操作层）→ cjc → .so + .java（互操作层）
5. 添加至您的 Android 项目：

   - 由cjc生成的.java文件
   - cjc生成的.so文件
   - 仓颉 SDK 提供的运行时库（.so 和 .jar）

   在Java源代码中插入互操作层所需的调用，并重新构建项目。

   Android项目 + .java, .jar, .so → Android工具链 → .apk

初始互操作类创建工作流和从 Cangjie调用Java部分通过端到端示例详细描述了上述过程。

初始互操作类创建工作流

步骤 1：设计互操作性层

在此步骤中，你*需从Java代码视角*设计一个或多个互操作类的API，即需为每个互操作类确定：

- 所属的Java包；
- 是否继承默认的 java.lang.Object 类（若非继承则需指定继承对象）；
- 将实现哪些Java接口（如有）；以及
- 该类将包含哪些公共/受保护的方法和/或构造函数。您只需确定参数类型，以及方法的名称和返回值类型；具体实现将在Cangjie中编写。

另请参阅互操作类的功能与限制。

示例：

假设我们希望通过调用某个静态方法 m() 将控制权从 Java 传递给 Cangjie，该方法接受类型为 com.example.a.A、java.lang.String 和 int 的参数，并返回类型为 com.example.b.B 的值。我们要求该方法属于名为 Interop 的类，该类继承自 Object 并属于 cjworld 包。

换言之，我们已确定在步骤4生成的Java源代码文件应包含以下内容：

```
package cjworld

import com.example.a.A;
import com.example.b.B;

public class Interop {
    /* 粘合代码 */
    public static B m(A a, String s, int i) {
        /* 调用Cangjie实现的m()方法的粘合代码 */
    }
    /* 更多胶水代码 */
}
```

步骤2：生成镜像类型声明

现在，您需要为互操作类所依赖的所有Java引用类型生成镜像类型声明：包括其超类型、方法和构造函数参数的类型等，以及这些类型的潜在依赖项。

注意：互操作库中提供了java.lang.Object和java.lang.String的镜像类，以及代表Java数组的泛型镜像类。若您的互操作类的方法和构造函数仅

向/从仓颉传递/接收的是Java基本类型、java.lang.Object、java.lang.String及上述类型的数组值。

使用仓颉 SDK 中的 Java 镜像生成器按以下方式生成镜像类型声明：

命令行：

```
java-mirror-gen \
    --package-name <包名> \
    --class-path <完整应用程序类路径> \
    --destination <输出目录> \
    <镜像类型名称>
```

或

```
/path/to/jdk/21/bin/java \ java-
mirror-gen \
    --package-name <包名> \
    --class-path <完整应用程序类路径> \
    --destination <输出目录> \
    -jar <jar-文件>
```

其中

- <package-name> 是所有镜像类型的仓颉包所需名称。
- <完整应用程序类路径> 是应用程序构建过程中使用的完整类路径，包含 android.jar。
- <输出目录> 是存放包含镜像类型声明的仓颉源文件的目录路径，例如 ./src/cj。
- <镜像类型名称> 是互操作类设计所依赖的 Java 引用类型的*完全限定名称*，不包括 java.lang.Object、java.lang.String 或数组。
- <jar-file> 是单个 jar 文件的路径名。其中包含的所有公共类和接口的 .class 文件都会被镜像，同时还会镜像 <完整应用程序类路径> 中发现的所有依赖项。

示例：

延续上述示例，我们注意到类 cjworld.Interop 依赖于：

- java.lang.Object作为父类；
- com.example.a.A、java.lang.String 和 int 作为参数类型；以及
- com.example.b.B 作为返回值。

我们无需镜像 int 等基本类型，而 java.lang.Object 和 java.lang.String 类已预先完成镜像。因此仅需镜像两个 com.example 类。假设我们希望它们出现在 javaworld 包中。

使用虚构应用程序类路径的镜像生成器命令行：

```
java-mirror-gen \
    --package-name javaworld \
    --class-path /home/user/Android/Sdk/platforms/android-35/android.jar:App.jar \
    --destination ./src/cj \ com.example.a.A
    com.example.b.B
```

这将生成文件 src/cj/javaworld/src/A.cj 和 src/cj/javaworld/src/B.cj，并为这两个 Java 类型可能存在的任何依赖项生成镜像类型声明。

。

步骤 3：编写互操作类

针对步骤1设计的每个Java类骨架，按以下方式编写新的仓颉类：

1. 使用适当的包名和类名（Java封装类的全限定名应保持一致）。
2. 导入 java.lang.* 包。
3. 若步骤二中生成过镜像类型，则导入所需镜像类型。当前暂不导入依赖项。
4. 为类添加@JavaImpl注解。
5. 使该类继承目标Java类的镜像（默认继承java.lang.Object的内置镜像）。
6. 使用 JObject、JString 和 JArray<T> 分别替代 java.lang.Object、java.lang.String 和 Java 数组类型。

类型映射 （T' 对应匹配的原始类型或对应的镜像类型）：

Java类型 (T)	仓颉码类型 (T')
boolean	Bool
byte	Int8
短整型	Int16
char	UInt16
int	Int32
long	Int64
float	浮点32
double	浮点64
Object	JObject 或 ?JObject
字符串	JString 或 ?JString
类 C	C' 或 ?C'

Java 类型 (T)	仓颉码 (T')
接口 I	I' 或 ?I'
T[]	JArray<T'> 或 ?JArray<T'>

对于可能接收/持有 Java null 的镜像类型和互操作类型的参数、返回值和局部变量，请使用 ?<T'>（Option<T'>）类型。

使用 Unit 作为返回类型来表示 Java 的 void 方法。另请参阅互操作类

的特性与限制。

示例：

延续上例，函数 m() 的互操作类定义如下：

```
package cjworld

import java.lang.* import
javaworld.*

@JavaImpl
public class Interop {
    public static func m(a: ?A, s: ?JString, i: Int32): ?B {
        /* 目前仅为占位实现 */
        B()         // 假设B类具有可访问的无参数构造函数
    }
}
```

若无需让 m() 向 Java 调用方返回 null，请将其 返回值类型中的 ? 移除。

步骤 4：编译互操作类

命令行：

```
cjc --output-type=dylib \
    -p <源代码目录> \
    -ljava.lang -linteroplib.interop \
    --输出Java生成目录=<java输出目录>
```

其中

<source-directory> 是包含互操作类源代码和镜像类型声明的目录路径名。

<java-输出目录> 是生成 Java 源文件的存放路径。

输出为包含互操作类编译后仓颉代码的.so文件，以及其Java封装类的Java源文件(.java)。

示例：

```
cjc --output-type=dylib \
    -p src/cj \
    -ljava.lang -linteroplib.interop \
    --输出Java生成目录=src/java
```

编译器将生成两个文件： /libcjworld.so 和 src/java/cjworld/Interop.java。

步骤5：整合所有内容

1. 将以下文件添加至您的Android项目：

   - 上一步生成的 Java 源文件——按包名对应放置于 src/main 下的源代码树中。
   - 上一步生成的 .so 文件 - 放置于 src/main/jniLibs/arm64-v8a（若子目录不存在则创建）。
   - 来自 $CANGJIE_HOME/runtime/lib/ 相应子目录的所有 .so 文件，例如：
     $CANGJIE_HOME/runtime/lib/linux_android31_aarch64_cjnative/
     - 同时复制至src/main/jniLibs/arm64-v8a目录。
   - Android NDK 中的 libc++_shared.so 文件 - 同样位于 src/main/jniLibs/arm64-v8a 。（该文件位于 *toolchains/llvm/prebuilt/host* /sysroot/usr/lib/aarch64-linux-android/ 位于Android NDK根目录下，其中host是对应构建系统操作系统和硬件的字符串，例如“linux-x86_64”。
   - 文件 $CANGJIE_HOME/lib/library-loader.jar 作为依赖项。

2. **重要提示：必须遵循传统规范，将所有 .so文件，否则应用程序在加载仓颉库时将崩溃。**

   在 Android 应用程序的 Gradle 构建脚本（通常名为 build.gradle.kts）中，定位 android {} 代码块，若尚未存在则插入以下选项设置：

```
android {
    packaging {
        jniLibs {
            useLegacyPackaging = true
        }
    }
```

```
}
```

3. 重新构建项目以确保当前所有内容正确无误。您也可以尝试部署该项目。

4. 添加与互操作类配合使用的Java代码，并重新构建您的Android项目。

示例：

现在可在Java代码中添加对Interop.m()的调用，将控制权传递至互操作类中的仓颉实现：

```
B b = Interop.m(new A(), "Test", 0);
```

从仓颉调用Java

完成前文所述互操作层的设计、构建与集成后，即可在互操作类的成员函数中添加使用Java类型的代码。类型映射规则保持不变：

仓颉类型 (T')	Java类型 (T)	备注
Bool	boolean	
Int8	byte	
Int16	short	
UInt16	char	
Int32	int	
Int64	long	
浮点32	float	
浮点64	double	
JObject 或 ?JObject	Object	
JString 或 ?JString	字符串	
T' 或 ?T'	T	(*)
JArray<T'> 或 ?JArray<T'>	T[]	(**)

(*) T' 必须是 Java 类型 T 的镜像类型，或为互操作类（其 Java 包装类 T 的源代码由 cjc 自动生成）。

(**) T' 必须是镜像类型、互操作类或上述值类型之一，例如 Int32。

使用 Unit 作为返回类型来调用 Java 的 void 方法。

限制：

- 不支持可变参数（varargs）方法和构造函数。

步骤 0：正常构建您的 Android 应用程序 步骤 1：生成镜像

类型声明

若仅需调用互操作类、其基类及/或已生成镜像声明的类型的成员函数（例如互操作类参数或返回值的类型），则可跳过此步骤。

命令行：

```
java-mirror-gen \
    --package-name <包名> \
    --class-path <完整应用程序类路径> \
    --destination <输出目录> \
    <镜像类型名称>
```

或

```
/path/to/jdk/21/bin/java \ java-
mirror-gen \
    --package-name <包名> \
    --class-path <完整应用程序类路径> \
    --destination <输出目录> \
    -jar <jar-文件>
```

其中

- <package-name> 是所有镜像类型的仓窩包所需名称。
- <完整应用类路径> 是应用构建过程中使用的完整类路径，包含 android.jar。
- <输出目录>是存放包含镜像类型声明的仓颉源文件的目录路径，例如 ./src/cj。
- <镜像类型名称> 是互操作类设计所依赖的 Java 引用类型的*完全限定名称*，不包括 java.lang.Object、java.lang.String 或数组类型。
- <jar-file> 是单个 jar 文件的路径名。其中包含的所有公共类和接口的 .class 文件均会被镜像，同时还会镜像 <完整应用程序类路径> 中发现的所有依赖项。

示例：

延续前节示例，假设我们希望函数 Interop.m() 调用某类的 String g(A a, int i) 方法

com.example.c.C，其中A是我们已镜像的类型com.example.a.A：

```
package com.example.c; import

com.example.a.A;

public class C {
    public static String g(A a, int i) {
        /* 返回字符串的Java代码 */
    }
}
```

我们重新运行镜像生成器，将 com.example.c.C 作为额外参数传递：

```
java-mirror-gen \
    --package-name javaworld \
    --class-path /home/user/Android/Sdk/platforms/android-35/android.jar:App.jar \
    --destination ./src/cj \
    com.example.a.A com.example.b.B com.example.c.C
```

此命令将生成包含镜像类型声明的相同文件，同时生成文件 src/javaworld/src/C.cj 以及该类型所有先前未镜像依赖项的镜像类型声明文件。

文件 src/cj/javaworld/src/C.cj 的内容如下：

```
package javaworld

import java.lang.*

@JavaMirror["com.example.c.C"]
public class C {
    public static func g(a: ?A, i: Int32): ?JString
}
```

步骤 2：导入镜像类型并编写代码

确保导入所有涉及的镜像类型，然后实现互操作类的构造函数和成员函数，将Java类型视为仓颉类型进行操作。

示例：

延续上一节的示例，我们可以使用镜像类型 javaworld.C 来实现函数 cjworld.Interop.m()：

package cjworld import

java.lang.*

```
import        javaworld.A
import        javaworld.B
import javaworld.C

@JavaImpl
public class Interop {
      public static func m(a: ?A, s: ?JString, i: Int32): ?B {let s1: JString =
            match (a) {
                  case Some(aa) => C.g(aa, i) ?? JString("") case None
                  => JString("")
            }
            B(s1)     // 假设存在 B(String) 构造函数
      }
}
```

步骤3：重新编译仓颉部分

具体步骤及示例请参阅互操作类创建流程的步骤4。

只要您在前面的步骤中没有更改互操作类的公共接口，只需更新 .so 文件即可。生成的 Java 源代码应与之前生成的完全一致。

步骤4：更新并重新构建Android项目

将上一步更新/生成的.so文件（如有必要也包括.java文件）复制到Android项目中，然后重新构建项目。

具体操作及示例请参阅互操作类创建工作流的步骤5。

互操作类的特性与限制

1. 互操作类*必须*是镜像类的直接子类。默认情况下，互操作类将继承镜像类java.lang.JObject，而非 std.core.Object。
2. 互操作类*可*实现一个或多个镜像接口，但绝不能实现常规的仓颉接口。反之，常规仓颉类型不得实现或继承镜像的Java接口。
3. 互操作类不得声明为公开或抽象类，且不可通过extend进行扩展。
4. 互操作类可引入*任意仓颉类型*的新实例字段，并可重写其镜像超类的成员函数。
5. 互操作类的构造函数可通过super()调用超类构造函数，但实例成员函数调用受限于常规仓颉构造函数的相同约束，且必须初始化所有新增成员变量。

，同时必须初始化所有新增成员变量。

6. 互操作类的实例成员函数可调用该互操作类从其镜像父类继承的实例成员函数，并/或使用super() 调用该互操作类覆盖的父类的成员函数。

7. 所有镜像类型和互操作类的构造函数及成员函数签名仅可使用以下类型的参数：(a) 镜像类型或互操作类本身，或 (b) 与Java基本类型完全对应的类型。详见《从仓颉调用Java》中的类型映射表。

8. 镜像类型与互操作类的值：

   - 不得逃逸至全局或静态的Cangjie变量，亦不得逃逸至此类变量所引用的任何数据结构。cjc编译器目前尚未强制执行此限制，因此需严格遵守编程规范。若未遵守该规范，可能导致程序异常终止。
   - 当不再需要时必须显式释放。最好在首次有机会时释放，但无论如何应在控制权返回JVM之前完成。否则可能导致内存泄漏，并引发JVM抛出OutOfMemoryError异常。

   详情请参阅Java引用管理章节。

以下三项限制为Android/JVM特有：

9. 任何使用镜像类型或互操作类型的仓颉代码，必须在Java虚拟机注册的线程中执行。该线程可由Java代码创建，也可通过Java调用API注册（即"附加"）至JVM的操作系统线程。此要求当前版本需通过编程规范强制遵守，未来可能由仓颉编译器强制执行。

10. 所有镜像类型和互操作类的Java对应实现必须由同一类加载器加载。

11. 与Java及其他JVM语言不同，Cangjie禁止包之间存在循环导入依赖关系。这给镜像生成过程带来挑战，详情请参阅"循环导入依赖处理"子章节（）。

Java到仓颉映射规则

当前版本的镜像生成器执行以下Java→仓颉转换规则。

通用注意事项

镜像生成器以*Java*类文件为输入。因此，任何Java编译器未从Java源代码传播到类文件的信息都无法被镜像生成器获取。最值得注意的是，这涉及泛型和方法参数的 名称。

名称

Java类型、字段及方法的原始名称将尽可能保留。若因下列原因无法保留原始名称，则通过注解传递至仓颉编译器。

- 与仓颉关键字冲突的Java标识符（如"func"、"main"或Int32）需用反引号``包裹，形成原始仓颉标识符：

  ```
  public static final long Int32 = 0xffff_ffff; public static let `Int32`: Int64
  ```

- Java标识符可能包含仓颉标识符中不允许的字符，最显著的是美元符号 $，该符号用于嵌套Java类型的二进制名称中。此类字符将被替换为下划线 _：

  ```
  public class Outer {
      public class Inner {}
      public Inner getInner() { return new Inner(); }
  }
  ```

  ```
  @JavaMirror["Outer"] public open
  class Outer {
      public init()

      public open func getInner(): ?Outer_Inner
  }

  @JavaMirror["Outer$Inner"] public open
  class Outer_Inner {
      public init(p0: ?Outer)
  }
  ```

- 尽管被视为不良实践，Java类型中字段、成员类型和方法仍可共享相同简单名称（标识符）。这种设计可行，因为名称含义始终可通过语法（仅方法名后可跟“()”）或上下文确定。此外，只要签名不同，Java中同一类型的实例方法和静态方法可使用相同名称。

  但在仓颉语言中，所有成员变量和非重载函数

必须具有唯一名称。（仓颉没有成员类型，因此成员类型名称冲突根本不会发生，因为它们会被映射到顶级类型。特别地，实例成员函数和静态成员函数不能具有相同名称。）

因此，镜像生成器会在实例变量名称后附加“*_type-name*”，并在静态方法名称后附加“Static”，以避免其原始Java名称与其他字段/方法名称冲突。它将原始名称保留为@ForeignName注解的值：

```
public class Node {
    public int id;
    public Node(int id) { this.id = id; }
    public static int id(long x) { return (int)x; } public static int
    id(short x) { return x; } public int id() { return id; }
    public void id(int newId) { this.id = newId; }
}
```

翻译为

```
public open class Node {
    @ForeignName["id"]
    public var id_Node: Int32

    public init(arg0: Int32)

    @ForeignName["id"]
    public static func idStatic(arg0: Int64): Int32

    @ForeignName["id"]
    public static func idStatic(arg0: Int16): Int32public open func

    id(): Int32

    public open func id(arg0: Int32): Unit
}
```

- Java包名无法在实践中保留，因为Java中普遍存在的包间循环导入依赖在 仓颉中是被禁止的。详情请参阅循环导入依赖处理。

基本类型

Java 基本类型映射至对应的仓颉值类型：

Java类型	仓颉类型
boolean	Bool
byte	Int8
短整型	Int16
char	UInt16
int	Int32
long	Int64
浮点数	浮点32
double	浮点64

类和接口

Java类和*接口定义*分别映射到注释为@JavaMirror的仓颉类和接口声明中。 该注解的值为字符串，保留镜像Java类型的原始全限定名。除非包含仓颉标识符中不允许的字符，否则镜像类型的简单名称将被保留。此类字符（最显著的是美元符号$，其在Java类文件名中的出现通常表示该类型为嵌套类型）将被替换为下划线_。

当Java类与接口类型作为镜像字段类型，或作为镜像方法/构造函数的参数类型及返回值类型时，将被包装为Option<T>（详见空值处理说明）。

@JavaMirror注解声明与常规仓颉类/接口定义存在以下差异：

- Java镜像类层次结构的根节点并非std.core.Object，而是内置镜像类java.lang.JObject
- 镜像 对应 ，java.lang.String 对应 ，同时 也为内置类， 其 名称为 java.lang.JString
- 仅镜像符号信息。变量初始化器及函数/构造函数主体被省略。

示例：

例如，以下 Java 类的镜像类：

```
public class Node {
public static final int A = 0xDeadBeef;
    private int id;
public Node(int id) { this.id = id; }
public int id() { return id; }
}
```

可能如下所示：

```
@JavaMirror["Node"] public
open class Node {
    public static let A: Int32public
    init(id: Int32)public func id():
    Int32
}
```

仅镜像公共Java类和接口。非final类通过open修饰符进行修改。sealed和non-sealed修饰符将被忽略，同样忽略的还有遗留修饰符strictfp。

私有成员、包私有成员、构造函数、静态初始化器和实例初始化器均不进行镜像。

若Java类型成员与其镜像名称因"命名规则"所述原因存在差异，则通过镜像成员上的@ForeignName注解值将原始Java成员名传递至Cangjie：

```
CurrencyAmount priceInUS$Per(WeightUnit wu) { ... } @ForeignName["priceInUS$Per"]
public open priceInUS_Per(arg0: WeightUnit): CurrencyAmount
```

注：访问修饰符protected在Java和Cangjie中的含义不同。

在 Java 中，允许从同一包内的任何位置以及其他包中定义的子类访问类的受保护成员或构造函数。

在 Cangjie 中，该修饰符还允许从子包和同一模块内的任何包进行访问。

通常情况下，这不会造成问题。

字段会被映射到对应镜像类型的成员变量中。只要字段名不与其他成员名称冲突，其名称将被保留。修饰符public、protected和static会被保留，而transient和volatile修饰符将被忽略。final字段会被映射为let成员变量，非final字段则映射为var成员变量。变量初始化器将被省略。

方法被映射为成员函数，其中参数类型和返回值类型由相应的镜像类型替代。void方法的镜像返回类型为Unit。实例方法名称得以保留。 静态方法名称只要不与实例方法名称冲突就会被保留。修饰符 public、protected 和 static 均被保留。修饰符 native 和 synchronized 以及旧式修饰符 strictfp 均被忽略。非 final 方法的镜像会添加 open 修饰符。

构造函数会被映射为初始化构造函数，其中参数类型会被替换为相应的镜像类型。*这包括默认构造函数——*

可能已隐式声明的默认构造函数。修饰符public和protected将被保留。

注：

1. 镜像类可能不具备初始构造函数。
2. 与传统仓颉类不同，不含初始化构造函数的镜像类不会隐式定义默认构造函数，因此无法通过调用构造函数实例化。对于自动生成的镜像类，构造函数缺失通常意味着被镜像的Java类仅声明了私有或包私有构造函数，以防止被任意代码实例化。另请参阅枚举类。
3. Java镜像生成器以*类文件*为输入，而类文件中通常缺少方法/构造函数参数的名称。此时，镜像生成器将为其分配名称 arg0、arg1 等。javac 编译器选项 -parameters 可强制保留参数名称，但仅适用于类，不适用于接口。调试信息生成选项 -g / -g:vars 同样如此。

成员类型会被映射到对应的顶级类型中，因为仓颉编码不支持类型嵌套。成员类型的映射名称由原始成员类型的*二进制*名称衍生而来，该名称由其直接包含类型的二进制名称、后缀美元符号 $ 以及成员类型本身的简单名称组成。 但由于仓颉标识符中不允许出现美元符号，故将其替换为下划线_（参见命名规范）。修饰符public和protected得以保留，而static修饰符将被忽略。镜像内置类的初始化构造函数会额外添加一个参数用于传递外部实例（该参数在Java中隐含）：

```
public class Outer {
    public static class Static {}
    public class Inner {}
    public Inner getInner() { return new Inner(); }
}
```

```
@JavaMirror["Outer"]public open
class Outer {
    public init()

    public open func getInner(): ?Outer_Inner
}

@JavaMirror["Outer$Static"]                    // 保留原始二进制名称 public open class
Outer_Static {                                 // '$' 替换为 '_'
    public init()
}
```

```
@JavaMirror["Outer$Inner"]                  // 保留原始二进制名称 public open class
Outer_Inner {                               // '$' 被替换为 '__'
    public init(p0: ?Outer)                 // 用于封装实例的额外参数
}
```

*所有*方法和构造函数的镜像中都省略了主体，因此它们在常规仓颉代码中都呈现为抽象成员函数。这导致以下语法变更：

- 类方法的抽象修饰符得以保留，否则将无法区分抽象与具体Java方法的镜像：

```
public abstract class A {
    public void c() {}
    public abstract void a();
}

@JavaMirror["A"]
public abstract class A {public
    init()

    public open func c(): Unit

    public open abstract func a(): Unit
}
```

- 出于相同原因，默认接口方法的镜像会标注 @JavaHasDefault。

```
public interface I {
    default void c() {} void
    a();
}

@JavaMirror["I"]public
interface I {
    @JavaHasDefault func
    c(): Unit

    func a(): Unit
}
```

在具有可变参数数量的方法中，... 将被忽略。

镜像类型继承

镜像类和接口形成独立的子类型层次结构，这意味着：

- Java镜像类层次结构的根节点并非std.core.Object，而是内置镜像类java.lang.JObject。
- 镜像接口可继承其他镜像接口，反映原始Java接口间的继承关系。镜像接口不可继承常规仓颉接口，反之亦然。
- 镜像类可继承其他镜像类，反映原始Java类的继承关系。镜像类不可继承常规Cangjie类，反之亦然。
- 镜像类可实现镜像接口，但不可实现常规Cangjie接口。特别地，它们不实现Any接口。常规 类不可实现镜像接口。

泛型

Java泛型在编译时会被擦除，因此自动生成的镜像类型始终为非参数化类型，其类型变量会被替换为左侧边界的镜像类型。

手动声明参数化镜像类型同样不可行。 类型JArray<T>（参见数组）是特殊例外。

数组

内置的特殊类型 java.lang.JArray<T> 用于表示 Java 数组的镜像。

类型为 *T* 的 Java 数组（*T* []）通常映射为：

- ?JArray<*T′*>（当T为基本类型时）
- ?JArray<?*T′*>，若 *T* 为引用类型

其中*T'*是*T*的镜像类型。关于Option<T>封装的原理，请参阅空值处理部分。

注意：Java数组具有协变性，而仓颉泛型具有不变性， 类JArray<T>亦不例外。

枚举类

Java枚举类*E*映射为仓颉镜像类*E′*，该类继承java.lang.Enum的镜像，自身既非开放类也非密封类，因此不可被扩展。

该类E'包含：

- *E*枚举常量的镜像，以*类型E'的*公共静态成员let变量形式存在。

- 不提供构造函数，以确保镜像类无法被实例化。（镜像类不具备默认构造函数。）
- 隐式定义方法 public static *E*[] values() 和 public static *E* valueOf(String name) 的镜像
- 镜像原始枚举类中所有 public/protected 字段与方法，视同普通类的字段与方法

特殊注意事项

空值处理

仓颉不支持空引用概念，因此没有与Java空类型对应的机制。若将Java方法返回值的引用类型直接映射至对应镜像类型，该方法返回空值可能导致段错误。反之，必须存在从仓颉向Java方法或构造函数传递空值的机制。

因此，若字段、数组元素、方法/构造函数参数或方法返回值的原始Java*类型为引用*类型R，则*该实体的*镜像类型为Option<*R'*>，*其中R'是类型R的镜像类型*。在仓颉编译器中，None表示空值，Some(*r'*)*则表示类型R'的*（非空）*引用值r'*。 cjc编译器将Option<T>识别为Java兼容类型，前提是T为镜像类型或互操作类，并据此对T的值进行包装/拆包操作。

例如，考虑以下 Java 接口：

```
interface Concatenator {
    String concat(String[] ss);
}
```

ss 参数本身可能为空，ss 数组的每个元素都可能为空，且 concat 方法可能返回空值。*因此*，安全地映射该接口的方式如下：

```
@JavaMirror
interface Concatenator {
    func concat(ss: ?JArray<?JString>): ?JString
}
```

基于相同原因，在互操作类代码中声明外部类型的局部变量时，除非能100%确定其不会被赋值为null，否则应使用Option<T>进行包装。

```
// 假设 M 是 Java 镜像类型
let m: M = M()          // 若 M() 返回成功，则保证
                        // 返回 M 的新实例
```

Option<T>包装确保了当Java世界中的空值潜入Cangjie世界时代码不会崩溃，但这会以性能和内存占用为代价。此方法的另一缺点是 会导致变体性丢失。

方差丢失

Java镜像类型和互操作类采用Option<T>包装时存在一个限制：此类包装类型在其他所有方面均遵循Cangjie的语义。具体而言，Option<T>*对其类型参数T具有不变性*：若U是T的子类型，则Option<U>并非Option<T>的子类型，除非U与T是相同类型。 对于镜像类型而言，这意味着任何依赖返回类型协变性的重写Java方法，在Option<T>封装下可能无法以相同方式镜像。其返回类型必须从超类方法继承而来。

示例：

假设Java类Foo是类Bar的直接父类：

```
public class Foo {}

public class Bar extends Foo {}
```

接口 C 声明了一个方法 get，该方法返回 Foo 的实例：

```
public interface C {
    public Foo get();
}
```

C 的子接口可重写 get 方法，使用更精确的返回类型 Bar：

```
public interface D extends C { @Override
    public Bar get();
}
```

若不使用Option<T>包装，所有类型可映射为：

```
@JavaMirror
public open class Foo {}

@JavaMirror
public open class Bar <: Foo {}

@JavaMirror
public interface C {
    public open func get(): Foo
}
```

```
@JavaMirror
public interface D <: C {
    public override open func get(): Bar                    // 返回类型协变的实际应用
}
```

但若 get() 可能返回 null，应用程序崩溃将不可避免。

使用Option<T>包装可确保安全，但所有重写方法的返回类型必须降级为原始方法的返回类型：

```
@JavaMirror
public open class Foo {}

@JavaMirror
public open class Bar <: Foo {}

@JavaMirror
public open interface C {
    public open func get(): ?Foo
}

@JavaMirror
public open interface D <: C {
    // public open func get(): ?Bar              // 错误，`Option<T>` 不是 T 的协变类型 public open
    func get(): ?Foo                             // 允许，但返回类型会被降低
}
```

空值标注可部分缓解此问题，但当前Cangjie SDK的 版本尚不支持该功能。

循环导入依赖处理

包之间的循环依赖不仅可能存在，在Java中更是无处不在。例如，无处不在的 java.lang.String类依赖于：

- java.io包中的Serializable接口，
- java.nio.charset 包中的 Charset 类，以及
- 以及来自 java.util 的 Locale 类，

而这三类当然都依赖于 java.lang 中的 Object 类！

Java通过编译为独立.class文件及延迟绑定机制有效规避了此问题。但要使仓颉导入声明编译通过，其导入的整个包必须已完成编译。因此，属于特定包的所有仓颉源文件必须在单次编译会话中合并编译为单一二进制文件。 可以说，苍颉编译的最小单位是整个包，而非单个文件。因此苍颉代码中不存在跨包的循环 导入依赖关系。

单包模式

上述两种语言的差异使得镜像生成过程中无法保留Java包名。实际操作中，镜像生成器必须将所有生成的镜像类型收集到单个Cangjie包中。该包的期望名称需通过选项 --package-name 选项指定该包的期望名称。

例如，若使用以下选项运行镜像生成器：

```
--package-name java.world
```

它将把所有生成的镜像类型声明放置在java.world包中，并通过@JavaMirror注解的参数将原始Java类型的全限定名传递给cjc编译器：

```
package java.world

import java.lang.*

@JavaMirror["java.lang.Cloneable"] public
interface Cloneable {
}
```

名称冲突

在不同包中声明的Java类型可能具有相同的简单名称，因为它们的完全限定名不同。例如，JDK的javax.management包中有一个Attribute类，而javax.naming.directory包中有一个Attribute接口。它们的镜像无法在同一个仓颉包中共存。 因此镜像生成器在单包模式下运行时会检测此类名称冲突，并对*所有*冲突名称进行处理。具体而言，它将用该类型的全限定名替代其简单名称，并将点号 . 替换为下划线 _：

```
//         src/java/world/src/javax_management_Attribute.cjpackage
java.world

import java.lang.*

@JavaMirror["javax.management.Attribute"]
public open class javax_management_Attribute <: Serializable {
    -    -    -

// src/java/world/src/javax_naming_directory_Attribute.cjpackage java.world

import java.lang.*
```

```
@JavaMirror["javax.naming.directory.Attribute"]
public interface javax_naming_directory_Attribute <: Cloneable & Serializable {
    - - -
```

增量镜像

将所有Java镜像类型放入单个仓颉包（如java.world）不仅会导致名称冲突，还会增加编译时开销且普遍不便。因此将这些大量声明拆分为多个包是极佳的选择。所幸并非所有Java包都处于单一导入依赖圈中，至少可以进行部分拆分。

首先，您可能会注意到，任何Android应用程序中都包含三组可能不存在循环依赖关系的包：

- 应用程序包
- Android API 包
- JDK API 包

这意味着即使在最坏的情况下，所有镜像类型也能分别隔离为三个仓颉包，代号分别为“app”、“android”和“java”。

此外，自Java 9起JDK API已实现模块化，Java模块间不存在循环导入依赖关系。因此在镜像过程中完全能够复现*JDK API模块的*结构。例如仓颉包java.base将包含来自java.base JDK API模块导出包的类型镜像：java.lang及其子包、java.io、java.math等。

最后，第三方库不应与其他应用程序组件和API存在循环导入依赖关系。若您的应用程序包含此类库，且您也需要在仓颉代码中使用它们，建议将每个库镜像到各自独立的仓颉包中。

*增量镜像*过程可实现此类分离。*针对每个目标仓颉包*（通过--package-name选项设置），需分别运行镜像生成器，每次需额外指定两个参数：

1. 需映射至该目标仓颉包的完整Java包列表。这些Java包之间可能存在循环导入依赖关系，也可能导入*先前*已映射的包/类型。
2. 所有*先前*镜像Java类型的完全限定名映射的累积列表。初始时该列表为空。

镜像生成器会为列表(1)中出现的包所属的所有公共类和接口生成镜像，并包含它们可能存在的任何依赖项(2)。随后，它将所有新建立的xml-ph-0000@deepl.internalJava→仓颉名称映射添加到列表(2)中。

未列入列表的依赖项（2）。随后将所有新建立的 Java→仓颉名映射追加至列表（2）。

命令行语法　增量镜像目前仅在单JAR模式下支持，这意味着必须使用镜像生成器的第二种命令行语法：

java-mirror-gen [*选项*] -jar *jar-文件选项必须包含*

以下内容：

--package-name *目标包名*

*目标包*名是目标仓颉包的名称。镜像生成器将把*所有*镜像类型放入该包中。

重要提示：每次以增量模式运行镜像生成器时，必须指定一个全新的、未曾使用过的仓颉包目标名称。 若首次运行镜像生成器时使用 `--package-name my.java.libs` 将某些类型镜像到 `my.java.libs` 包中，随后在未更改 `--package-name` 参数值的情况下针对不同输入再次运行，将导致镜像结果完全失效且出现不一致。

> 某种意义上，目标仓界包是镜像的最小单位，正如在仓界开发中源仓界包是编译的最小单位：你无法单独编译属于同一包的两个仓界源文件。

--package-list　*路径名*

*路径名*必须指向一个纯文本文件，该文件包含一组Java包名称。具体而言，该文件的每行必须是一个完全限定的包名称，可选地跟随通配符.*：

```
com.example.model com.example.ui.*
```

注意：若存在通配符，该条目将匹配指定包的所有子*包及*该包本身。此行为与Java及Cangjie导入语句中通配符的语义不同。

生成器将为列出的包中定义*的所有*非匿名非私有类型及其所有依赖项生成*镜像*，但已存在于导入映射文件中的依赖项除外（如有）。所有这些镜像类型将被放置在通过 --package-name 选项指定的单个仓颉包中（参见上文）。

此选项仅可在单JAR模式下使用。

--import-mappings *import-mappings-file*

import-mappings-file 是*导入映射文件的*路径名，该纯文本文件会累积生成所有镜像类型对应的 Java 到仓颉全限定名映射集。 镜像生成器在启动时读取该文件，并跳过已存在映射关系的Java类型——默认认为这些类型已在先前调用中完成镜像。若镜像过程无误完成，生成器会将所有新生成镜像的映射关系添加至映射集，并在终止前写入 ./imports_config.txt文件。

注意：若将 *import-mappings-file* 设置为 ./imports_config.txt，该文件将被覆盖。若需保留中间映射集（例如用于调试 相关功能），请在镜像生成器每次增量运行后备份该文件。

示例：JDK API镜像生成

如前所述，JDK API采用模块化设计。可通过分析模块列表及其依赖关系图，收集导出包清单，并使用以下命令行模板运行镜像生成器：

```
java-mirror-gen \
    --android-jar /path/to/android.jar \
    --package-name java.base \
    --package-list ./java.base.txt \
    --import-mappings ./imports_config.txt \
    --destination src/cj \
    --class-path /路径/到/android/SDK/平台/文件/目录 \
    -jar /路径/到/android.jar
```

文件 ./java.base.txt 应包含从 java.base JDK API 模块导出*供公共使用的*包列表：

```
java.io java.lang
java.lang.annotation
...
```

且文件 ./imports_config.txt 应为空或不存在。当上述命令成功执行后：

- 目录 ./src/cj/java/base/src 将包含仓颉源文件，其中为 ./java.base.txt 所列包中定义的所有公共类和接口声明了镜像类型，且
- 文件 ./imports_config.txt 将包含所有这些 Java 类型的全限定名与其在 java.base 包中仓颉镜像名称的映射关系，供后续镜像生成器调用时使用。

注意：java.base模块导出的许多类型*实现*依赖于跨JDK包定义的类和接口
例如 sun.util.locale.provider.LocaleDataMetaInfo 或 jdk.internal.misc.Unsafe。但镜像类型*声明*
；它们既不暴露对应Java类型的任何私有成员，也不涉及公共方法或构造函数的
实现细节。因此镜像生成器完全不会处理此类内部包的类文件。

现在我们可以选择一个仅依赖于java.base模块的JDK API模块（例如java.xml），并再次运行镜像生成器，只需修改两个选项的值：

- 将 --package-name 的值设为 java.xml
- 将 --package-list 的值设为包含从 java.xml JDK API 模块导出包列表的文件路径：

```
javax.xml.*
org.w3c.dom
```

请注意，可以在包名后添加通配符 .* 来同时表示该包及其所有子包。在上文示例中，javax.xml.* 既匹配 javax.xml 本身，也匹配 javax.xml.catalog、javax.xml.transform、javax.xml.transform.stream 等子包。

```
java-mirror-gen \
    --android-jar /路径/到/android.jar \
    --package-name java.xml \
    --package-list ./java.xml.txt \
    --import-mappings ./imports_config.txt \
    --目标目录 src/cj \
    --class-path /android/SDK/platform/files/目录路径 \
    -jar /路径/到/android.jar
```

./src/cj/java/xml/src 目录现应包含所有 java.xml 模块导出项的镜像声明，而 ./imports_config.txt 文件则应包含 java.base 与 java.xml 仓颉包的映射关系。

对剩余模块重复此过程，顺序可任意，但需确保每个模块的全部依赖项完成镜像后再镜像该模块本身。

增量镜像的使用

只要应用程序代码中需要从仓颉库调用的任意组件之间不存在循环导入依赖关系，即可使用增量镜像技术进行分离。特别是，每个第三方

Java库的公共类和接口均可单独镜像到命名恰当的Cangjie包中。

实现方法如下：首先将构成应用程序的包集分解为彼此间无循环导入依赖的子集，然后采用上述增量镜像流程：

1. 选择任何不依赖其他子集包的子集，将其整体镜像到一个命名恰当的包中。
2. 镜像任何仅依赖于零个或多个先前镜像子集的子集。
3. 重复前一步操作，直至所有需用于仓颉编码的Java类型均完成镜像，*每次选择新的目标仓颉包 名称*。

闭包深度限制

除了包之间大量存在的循环导入依赖关系外，标准Java API和许多流行的Java库及框架还具有另一个固有特性：巨大的导入闭包规模。

例如，调用镜像生成器为空枚举类型生成镜像

```
public enum E {}
```

会同时生成java.lang.Enum*及其所有*标准库*依赖项*的镜像，总计约300个镜像类型：

AbstractInterruptibleChannel.cj
AbstractStringBuilder.cj
AccessControlContext.cj AccessMode.cj
AccessibleObject.cj

- - -
ZoneOffsetTransitionRule.cj
ZoneRules.cj ZonedDateTime.cj

对于所有这些镜像类型，编译器都会生成粘合代码，其中大部分在实际程序中永远不会被使用。这种情况在桌面或服务器环境中或许尚可容忍，但在Android等移动平台上则不可接受。

要求开发者提供精确的镜像类型和成员列表过于苛求。所幸通过在计算依赖闭包时简单*限制深度*，即可获得相当理想的效果：生成的镜像数量可减少一个数量级甚至更多。例如，将深度限制为2来镜像上述空枚举类型E时，仅会产生六个镜像类型：

除类型E本身的镜像外：枚举基类java.lang.Enum、其实现的三个接口，以及其中两个方法的返回类型：

Class.cj
Comparable.cj
Constable.cj E.cj
Enum.cj Optional.cj
Serializable.cj

深度限制的具体工作原理如下：

- 镜像*类型*集始终包含：
  - 原始Java类型，映射为对应的仓颉值类型；
  - 镜像生成器命令行中显式指定的类与接口类型；
  - java.lang.Object 与 java.lang.String（已分别预映射为 JObject 和 JString）；以及
  - 数组类型（其元素类型属于上述类型之一）映射为JArray<T>。
- 随后对集合中包含的类型*成员*进行如下过滤：
  - 即使相应超类型未包含在镜像类型集合中，继承的成员*也不会*作为“自有”成员进行镜像。
  - 字段类型未包含在镜像类型集合中的字段将被省略。
  - 签名中包含未被镜像类型集包含的类型的方法将被省略。
  - 成员类型与顶级类型相同：除非在镜像生成器命令行中明确指定，否则仅当它们在本文所述的有限深度依赖闭包计算过程中被包含在镜像类型集合中时才会被镜像。
- 命令行选项 --closure-depth-limit 用于设置显式指定的类和接口类型的深度限制。
- 若类型 *T* 的深度限制为零，*则其所有依赖类型均不会加入映射*集，甚至 *T* 的超类型也不例外。

示例：

```
// A.java
public class A {
    public void f(String s) {}
}
```

```
// B.java
public class B extends A {
    public void g(String s) {}
}
```

若镜像生成器被指示对类B进行深度限制为0的镜像处理，则仅会为B及其方法g(String)生成镜像，因为JString会自动包含在镜像类型集合中。同时，B从A继承的方法f(String)将不可用：

```
// B.cj
@JavaMirror["B"] public
open class B {
    public init()

    public open func g(s: ?JString): Unit
}
```

- 若给定类型 *T* 的深度限制为任意*正整数* N，则镜像集合*额外*包含以下类型，其深度限制为 *N* - 1：
  - *T* 的所有（即递归收集的）超类型；
  - *T本身声明*的非私有字段类型（即非继承字段类型）；
  - *T* 的非私有构造函数的参数类型；以及
  - *T本身声明*的非私有方法的参数类型和返回值类型。需再次注意：继承的方法*不会*被扫描。

若某类型已存在于镜像集中但深度限制较低，则将其深度限制更新*为N* -1。

该过程将递归地对镜像集合中新增的所有类型重复执行。

示例：

```
// A.java
public class A {
    public void f(C c) {}
}

// B.java
public class B extends A {
    public void g(D d) { }
}

// C.java
```

```
public class C { }

// D.java
public class D extends C {}
```

若镜像生成器被指示对类 B 进行深度限制为 1 的镜像处理，则会为 B、其父类 A 以及作为 B.g(D) 方法唯一参数类型的 D 生成镜像。但 A 和 D 的镜像深度限制将设为 0，因此类 C 及其方法 A.f(C) 不会被镜像：

```
// A.cj
@JavaMirror["A"] public
open class A {
    public init()
}

// B.cj
@JavaMirror["B"]
public open class B <: A {public
    init()

    public open func g(d: ?D): Unit
}

// D.cj
@JavaMirror["D"] public
open class D {
    public init()
}
```

最后，将深度限制设为2将使镜像生成器在其输出中包含 类C的镜像以及方法A.f(C)的镜像。

Java镜像生成器参考

先决条件

Java镜像生成器运行需安装JDK 17。若未安装JDK 17或未将其添加至可执行文件搜索路径，请在使用前完成安装及路径配 置 。

您还需知晓待镜像类所有依赖项所在的jar文件及目录的*本地*路径。这包括

Android标准库jar文件，以及应用启动时出现在类路径 中的jar文件。

命令行语法

Java镜像生成器可通过两种方式运行：

java-mirror-gen [*选项*] [*类型名称*]

生成一个或多个 Java 类和/或*接口及其*所有依赖项的镜像，*或*

java-mirror-gen [*选项*] -jar *jar-file* *（单JAR模式）*

生成*指定jar文件*所含.class文件对应的所有类型及其依赖项的镜像（若存在于指定类路径的其他位置）

其中

- 选项为镜像生成器的命令行参数。
- *类型*名称是需要镜像的Java类和接口的*完全限定名称*（即*不是*其.class文件的路径名）。
- jar-*file* 是单个 jar 文件的路径名。

注意：(增量镜像和/或闭包深度限制可能在每次调用镜像 生成器时限制镜像类型及其成员的集合。

命令行选项

- -a *路径名*, -android-jar *路径名* *(必选)*

  *路径名*必须指向用于构建应用程序Java部分的Android SDK中的android.jar文件。此选项必须始终存在且指向有效文件，否则脚本将报错失败。

- -d *目录*, --destination *目录*

  *目录*是镜像生成器放置其输出文件的路径名，这些文件包含生成的镜像类型声明，采用符合CJPM要求的子目录层次结构存储。若未指定此选项，则默认使用当前目录。

- -cp *路径*, --class-path *路径*

  *路径*参数为目录、jar文件及zip文件的路径名列表，在Windows系统中以分号； 分隔，其他平台则使用冒号： 分隔。镜像生成器将遍历这些目录及归档文件，

查找命令行指定的Java类型（*类型名*）及其依赖项的类文件时，遍历这些*目录和归档文件*。

- -p *name*, --package-name *name*　　*（必填？）*

  name 是所有生成的镜像类型将被放置的仓颉包名称。详情请参阅单包模式。

- -c *number*, --closure-depth-limit *number*

  number 是限制依赖关系图扫描深度的非负十进制整数，用于确定将镜像的类型及其成员集。

  详见闭包深度限制说明。

- -jar *jar-file*

  jar-*file* 是以“单JAR”模式处理的JAR文件路径名（参见上文命令行语法）。

- -l *路径名*, --package-list *路径名*

  pathname 是包含包名列表的纯文本文件路径名。仅镜像属于列出包及其依赖项的类文件。需配合 -jar 使用。详情参见增量镜像部分的命令行语法说明。

- -i *路径名*, --imports *路径名*

  pathname 是包含镜像类型映射的纯文本文件路径，该映射在先前镜像生成器运行中累积生成。需配合 -jar 使用。详见命令行语法中增量镜像部分。
  -l 和 -jar。 详情请参阅增量镜像部分中的命令行语法。

- -h, -?, --help

  显示包含命令行语法简要说明及所有选项的帮助信息后退出。

- -v, --verbose

  指示镜像生成器输出当前操作状态信息。

示例

```
java-mirror-gen \
    -android-jar $ANDROID_SDK/platforms/android-35/android.jar \
    -class-path ./classes \
    -destination ./mirrors \
    -p com.example \
    com.example.子包1.A com.example.子包2.B
```

```
java-mirror-gen \
    -a $ANDROID_SDK/platforms/android-35/android.jar \
    -cp ./lib \
    -d ./mirrors \
    -p javaworld \
    -jar App.jar
```

更复杂的示例请参见示例：JDK API镜像。

互操作库 API 参考

Android互操作库包含两个基础Java类（java.lang.Object和java.lang.String）的增强型镜像类型，以及用于表示Java数组的泛型镜像类型。

为支持使用未限定名称，这些镜像类型分别命名为JObject、JString和JArray<T>，以避免与std.core库中仓颉类型的名称冲突。

它们还重命名了某些方法并添加了新方法，以增强与 的互操作性。

java.lang.JObject

java.lang.JObject 是 Java 镜像类和互操作类层次结构的根类。它本身是 java.lang.Object 类的镜像，但删除了某些不支持的方法，并重命名和添加了某些方法，以更好地与仓颉标准库保持一致。

注意： 从JObject移除的方法在其他镜像类中同样不可用，包括clone()、finalize()和getClass()。

```
package java.lang

@JavaMirror["java.lang.Object"] open
class JObject {
    public open func equals(obj: ?JObject): Bool

    public func hashCode(): Int64
    @ForeignName["hashCode"]
    public open func hashCode32(): Int32

    public func toString(): String
    @ForeignName["toString"]
    public open func toJString(): JString

    public func wait(timeoutMillis: Int64): Unit
    public func wait(timeoutMillis: Int64, nanos: Int32): Unit
```

```
    public func wait(): Unitpublic func
    notifyAll(): Unitpublic func
    notify(): Unit
}
```

实例成员函数 equals 以及所有 wait/notify 函数均对应于相应的 Java 方法。

public func hashCode(): Int64

调用原始Java方法hashCode，并将返回的32位整数值强制转换为Int64类型。此举旨在更好地满足仓颉开发者的预期。

```
@ForeignName["hashCode"]
public open func hashCode32(): Int32
```

原始Java方法hashCode，为避免名称冲突重命名为hashCode32。

public func toString(): String

调用原始Java方法toString，并将结果转换为仓颉字符串。若底层Java方法返回null（极罕见情况），则抛出异常。

为更好满足仓颉开发者的预期而引入。

注意：此函数返回仓颉字符串，违反了镜像类型和互操作类中公共成员函数及构造函数参数与返回值必须使用Java兼容类型的要求。此行为仅因cjc编译器对此成员函数提供专属支持而得以实现。

```
@ForeignName["toString"]
public open func toJString(): JString
```

原始Java方法toString，为避免名称冲突重命名为toJString。

Java toString的实现极少返回null，因此 的返回类型为JString而非?JString。

java.lang.JString

```
package java.lang

@JavaMirror["java.lang.String"]open class
JString {
        -   -   -
    public init(cjString: String)
        -   -   -
}
```

public init(cjString: String)

将给定的仓颉字符串有效转换为Java字符串（JString）。

注意：此构造函数接受仓颉字符串类型的参数，这违反了镜像类型和互操作类中非私有成员函数及构造函数的参数与返回值必须使用Java兼容类型的要求。此行为仅因cjc编译器对JString提供了专属支持而得以实现。

从 JObject 继承的成员： equals、 hashCode、 hashCode32、 toString、 toJString、 wait/notify方法。

java.lang.JArray<T>

JArray<T> 是特殊内置泛型镜像类型，可映射所有 Java 数组类型。T 必须是映射至原始 Java 类型的值类型（如 Int32 或 Bool）、镜像类型或互操作类。

限制：

当前版本限制：

- 不支持包含可为空元素的Java数组。换言之，若T为镜像类型或互操作类，则支持JArray<T>类型，但不支持JArray<?T>类型。
- Java数组类型的变量和参数不能使用Option枚举标记为可为空。也就是说，即使JArray<T>是有效类型，?JArray<T>也不会被识别为该类型。

相较于仓颉数组<T>结构体类型，Java数组的API功能有限；除构造外仅支持长度查询、元素访问及java.lang.Object继承的方法。

public init(length: Int32)

构造指定长度的全新Java数组。 public prop length:

Int32

数组中的元素数量。

public operator func [](index: Int32): T
public operator func [](index: Int32, value!: T): Unit 元素访问运算符 [].

从 JObject 继承的成员：equals、hashCode、hashCode32、toString、toJString、wait/notify 方法。

简介

仓颉多平台技术使仓颉开发者能够将代码集成到新的Android/iOS应用程序中，并逐步用仓颉代码替换现有应用程序中的Java/Objective-C部分。实现跨语言、跨运行时互操作的核心机制是*镜像类型*，它将一种语言的类型暴露给另一种语言。

在仓颉侧，镜像类型支持通过方法覆盖继承Java/Objective-C类，同时实现Java接口和Objective-C协议，全程采用标准仓颉语法。而在Java/Objective-C侧，代表仓颉类型的镜像类型则以对应语言自身的类型形式呈现。 总而言之，这使得应用程序的Java/Objective-C部分与仓颉部分之间能够无缝衔接，包括在仓颉代码中使用目标操作系统的API。

挑战与解决方案

Java、Objective-C和仓颉均是支持继承与多态的面向对象语言。然而，三者在语义、对象模型及执行机制上的差异，使得仓颉对象无法直接用于Java或Objective-C代码中，反之亦然。

那么，这三种语言都拥有支持自动内存管理、线程处理、异常处理及其他底层功能的运行时系统，但实现方式各不相同。让两个独立设计的复杂语言运行时相互感知，可能会使整个系统的复杂度超出人类理解范围。

相反，仓颉与Java的互操作性是通过将二者伪装成低级语言来实现的。应用程序中的仓颉部分和Java部分通过Java本机接口（JNI）相互感知——该接口最初设计用于促进C和C++等语言开发Java本机方法。JNI虽功能丰富却属于低级API，而编写Java本机方法向来以繁琐著称。 所幸CJMP自动化消除了JNI的复杂性。

在iOS平台上，Objective-C运行时模块API承担着类似功能。它同样专为构建Objective-C与其他语言间的桥梁层而设计。与JNI相同，CJMP也承担了繁琐部分的处理工作。

额外设置

当前版本的iOS版仓颉SDK需要您从仓颉开源库下载Cangjie.h头文件：

https://gitcode.com/Cangjie/cangjie_runtime/blob/dev/runtime/src/Cangjie.h

并将其集成到你的 XCode 项目中，以便使用本文档所述的互操作性功能。

关键概念

镜像类型

将仓颉和 Objective-C 视为一对互操作语言，在其中一种语言中定义的*镜像类型 T'* 代表另一种语言中定义的现有类型 *T*，使第一种语言编写的代码能够使用该类型（可能存在某些限制）。

两语言中本质相同的布尔类型和数值类型会自然镜像对应：Cangjie类型Int32的Objective-C镜像类型为int，反之亦然。目前无法镜像另一语言不原生支持的数值类型，例如Cangjie类型Float16无法映射到Objective-C基本类型。

对于类、结构体、协议或接口等用户定义类型，映射类型即为另一语言中最接近的等效类型。例如，唯一能以合理精度近似Cangjie结构体类型的Objective-C类型，是附加objc_subclassing_restricted属性的Objective-C类。

镜像类型会暴露原始用户定义类型的成员与构造函数，使其在另一语言中可*访问并*使用。因此，返回Float16类型的Cangjie函数仍无法直接映射至Objective-C。

通常情况下，您可以在另一种语言中自动获取所需类型的镜像类型定义及其依赖项。对于Objective-C类型，系统提供了独立的镜像生成器；而Cangjie类型的Objective-C镜像则由cjc编译器在常规编译过程中作为副产品生成（）。

将Objective-C类型映射至Cangjie

cjc编译器会将所有Objective-C镜像类型的使用替换为相应的粘合代码，因此仅类型名称及其可访问成员的名称和类型具有实际意义。因此Objective-C类型的镜像仅包含成员声明，不含具体定义：构造函数与成员函数/属性的主体被省略，成员变量不包含初始化器。基于相同原因，@private成员也将被忽略。

要使这种对常规仓颉语法的扩展生效，每个镜像类型都必须标记为@ObjCMirror注解。这有助于编译器区分镜像类型声明与常规仓颉类型定义。

例如，针对以下 Objective-C 类的镜像类：

```
@interface Node : NSObject {
}
- (id)initWith:(int)x;
- (int)getX;
@end
```

可能如下所示：

```
@ObjCMirror
public open class Node <: NSObject {
    @ForeignName["initWith:"]
    public init(x: Int32)
    public open func getX(): Int32
}
```

将仓颉类型映射到Objective-C

与Objective-C类型的镜像不同（cjc对此提供专属支持），仓颉类型的镜像必须在外观上完全符合常规Objective-C类型，才能被iOS工具链识别。因此cjc编译器会以Objective-C源代码文件的形式生成这些镜像，其中包含负责连接Objective-C与仓颉世界的粘合代码。

例如，若前文示例中的 Node 类在仓颉中原始定义如下：

```
public class Node { private let
    _x: Int
    public func x(): Int { _x }
    public init(x: Int) { this._x = x }
}
```

那么cjc编译器可为其生成如下Objective-C镜像代码（为简洁起见，粘合代码基本省略）：

```
// Node.h
@interface Node : NSObject
- (id)init:(int64_t)x;
- (int)x;
@end

// Node.m
@implementation Node
- (id)init:(int64_t)x  {
    /* 粘合代码：构造仓颉节点实例并关联
     * 它与正在构造的 Objective-C Node 实例相关联， 即 'self'。
     */
}
- (int64_t)x {
```

```
    /* 胶水代码：调用关联的 Cangjie Node 实例的 'x' 成员函数
     * Cangjie Node 实例的 'x' 成员函数并返回结果。
     */
    }
@end
```

镜像函数

Objective-C 和仓颉均支持顶级全局函数，此类函数不属于任何其他类型的成员。它们以*镜像函数*的形式暴露给另一种语言，本质上是自动生成的粘合代码片段，用于在语言间传递控制权和数据。

互操作类

*互操作类*本质上是从一个或多个镜像类型派生而来的仓颉类，可在Objective-C中使用。其所有构造函数和非继承的公共成员函数均通过共轭*包装类*暴露给Objective-C代码，该包装类由cjc编译器自动生成。 封装类本身不定义其他用户可调用的方法或构造函数，但其从父类型继承的任何方法均可从Objective-C和Cangjie代码中调用。

例如，编译以下 Objective-C 互操作类时：

```
@ObjCImpl
public class BooleanNode <: Node { private let
    _flag: Bool
    public init(x: Int32, flag: Bool) {
        super.init(x)
        this._flag = flag
    }
    public func flag(): Bool {
        _flag
    }
}
```

cjc编译器还将生成一对类似于以下的Objective-C源代码文件：

```
// BooleandNode.h
@interface BooleanNode : Node
/* 粘合代码 */
- (id)init:(int32_t)x:(BOOL)flag;
- (BOOL)flag;
/* 更多粘合代码 */
@end
```

```
// BooleandNode.m
@implementation BooleanNode : Node
/* 粘合代码 */
- (id)init:(int32_t)x:(BOOL)flag    {
        /* 胶水代码构造 Cangjie BooleanNode(x, flag) 实例并
         * 将其与正在构造的Objective-C实例关联起来，
         * 即'self'。
         */
}
- (BOOL)flag {
        /* 胶水代码调用与'self'关联的仓颉
         * BooleanNode实例的'flag'成员函数并返回结果。
         */
}
/* 更多胶水代码 */
@end
```

外部类型

镜像类型和互操作类并非完全属于其定义语言的原生类型。延续比喻，它们的地位更像是临时访客和工作签证持有者，因此本文档中统称为*外部类型*。

Objective-C兼容类型

以下仓颉字符集类型被称为*与Objective-C兼容*：

- 在Objective-C基本类型中存在直接对应的值类型（包含Int16，但不包含Float16）
- 外部类型
- Option<*T*> 形式的类型，其中 *T* 为外部类型（参见*空值处理*部分的说明）

出于显而易见的原因，外部类型的公共成员函数参数和返回值只能采用与对应语言兼容的类型。

互操作类的成员变量可具有任意类型。其具有Objective-C兼容类型的公共成员变量可同时从Objective-C和Cangjie访问。

互操作场景

Objective-C与Cangjie的交互场景主要分为两类：

- 一种场景是将仓颉类型和函数暴露给Objective-C，使Objective-C代码能够使用仓颉的API、库及应用组件。

- 在另一种场景中，Objective-C类和接口被暴露给仓颉，使仓颉代码能够访问iOS API、第三方Objective-C库以及应用程序中仍以Objective-C编写的组件。

上述场景*并非*互斥，单个应用程序亦可同时采用两种模式。此外，两种场景均支持类继承与接口实现（Objective-C类可继承仓颉类，反之亦然）。这使得回调机制无缝衔接，因此两种模式均非单向操作。但每种模式在功能集、限制条件、工具支持等方面存在差异，故需分别阐述。

在Objective-C中使用仓颉

要在 Objective-C 中访问仓颉库、API 或其他代码片段，需为需要暴露的仓颉类型生成 Objective-C 镜像类型。cjc 编译器可完成此操作，具体方法详见《仓颉镜像生成参考》。

注意：仓颉语言的类型系统比Objective-C更灵活，且两者共享特性存在根本差异，最显著的是泛型支持。因此某些仓颉特性无法在Objective-C中实现，其他特性也难以自然紧凑地表达。具体支持特性及限制请参阅《仓颉到Objective-C映射》。

示例：

假设您希望向Objective-C暴露一个定义如下、名为Vector的Cangjie结构体：

```
package cj

import interoplib.objc.*

public struct Vector {    private
    let _x: Int32    private let
    _y: Int32

    public prop x: Int32 { get() { _x } } public prop y:
    Int32 { get() { _y } }

    public init(x: Int32, y: Int32) {
        _x = x
        _y = y
    }
```

```
    public func add(v: Vector): Vector { Vector(x + v.x,
        y + v.y)
    }
}
```

首先，你需要在定义 Vector 的源代码文件中导入 interoplib.objc 包：

```
import interoplib.objc.*
```

在编译 Vector.cj 时，需向 cjc 编译器添加两个额外选项：--experimental --enable-interop-cjmapping=ObjC。编译器将生成两个附加文件 Vector.h 和 Vector.m，内容类似如下：

```
// Vector.h
#import    <Foundation/Foundation.h>
#import    <stddef.h>

__attribute__ ((objc_subclassing_restricted))
@interface Vector : NSObject

/* 粘合代码 */

- (id)init:(int32_t)x :(int32_t)y;@property

(readonly, getter=x) int32_t x;
- (int32_t)x;
@property (readonly, getter=y) int32_t y;
- (int32_t)y;

- (Vector*)add:(Vector*)v;

/* 粘合代码 */

@end
// Vector.m

/* 粘合代码 */

@implementation Vector

/* 粘合代码 */

- (id)init:(int32_t)x:(int32_t)y    {
                                    /* 胶水代码构造仓颉向量实例(x,y)并关联
```

```
        * 关联到'self'。
        */
}
- (int32_t)x {
    /* 粘合代码获取关联的仓颉向量实例的'x'属性值
        * 关联的仓颉向量实例的'x'属性值。
        */
}
- (int32_t)y {
    /* 胶水代码，用于获取关联的仓颉向量实例的'y'属性值
        * 关联的仓颉向量实例的'y'属性值。
        */
}
- (Vector*)add:(Vector*)v {
    /* 调用与'self'关联的仓颉向量实例的'add'成员函数的粘合代码
        * 实例调用'add'成员函数， 传递'v'关联的仓颉向量
        * 实例， 并将结果封装为新的
        * Objective-C Vector类的实例。
        */
}

/* 更多粘合代码 */

@end
```

现在您可以将 Vector.h 和 Vector.m 添加到 XCode 项目中，并在 Objective-C 代码中像使用普通 Objective-C 类一样使用 Vector 类：定义 Vector* 类型的变量、创建实例、传递实例（包括通过 [Vector add] 传递给仓颉代码）等。

重要提示：当前版本的iOS版仓颉SDK还需从仓颉开源库下载Cangjie.h头文件并集成至XCode项目：

https://gitcode.com/Cangjie/cangjie_runtime/blob/dev/runtime/src/Cangjie.h

限制暴露范围

选项 --enable-interop-cjmapping 会指示 cjc 编译器为其编译的所有公共仓颉类型生成镜像类型，并暴露这些类型*的所有*公共成员函数和构造函数。然而，这种完全暴露往往并不理想，因为您在 Objective-C 中实际需要使用的功能，可能仅占仓颉库或框架所提供功能的一小部分。

您可通过向cjc传递专用配置文件的路径名来精确控制仓颉类及其成员的可见性，使用选项

--import-interop-cj-package-config-path 向 cjc 传递专用配置文件的路径名，精确控制仓颉字包及其成员的暴露范围。该文件为纯文本文件，内容必须符合 TOML 语法规范。

在文件的[default]部分，可为所有仓颉包指定默认API暴露策略，随后可在对应的[[package]]条目中针对特定仓颉包进行修改。例如：

```
[default]
APIStrategy="None"                # 默认不暴露任何接口

[[packages]]
name="com.example.pkg1"           # 从此特定包开始，
API策略="完整"                    # 暴露所有内容 # .
      -   -
```

此外，对于每个包，您可以使用included_apis或excluded_apis属性分别暴露或隐藏特定类型和/或成员：

```
#     -   -   -
[[packages]]
name="com.example.pkg2"                 # 从此特定包中，
included_apis = [ "Vector",             # 仅包含类型 'Vector'
                  "Vector.add"          # 及其成员函数 'add'
                ]                       # 暴露

[[packages]]
name="com.example.pkg3"                 # 从此特定包开始，
API策略="完整"                          # 所有内容
excluded_apis = [ "TopSecret",          # 但类型 'TopSecret' 和
                  "Auth.getPwd"         # 成员函数 'Auth.getPwd'
                ]                       # 暴露
```

更多详情请参阅仓颉镜像生成参考。

通用仓颉类型映射

许多重要的仓颉类型是参数化的。遗憾的是，仓颉与 Objective-C 通用类型的某些根本差异使得前者无法映射到后者。作为变通方案，仓颉 SDK 支持镜像类型的*单态化*：它为给定参数化仓颉类型的每组有效类型参数生成独立*的非通用* Objective-C 镜像类型。

注意：当前版本仅支持使用基本类型作为类型参数实例化的泛型类型镜像。

例如，若存在泛型仓颉类 p.Pair<T,U>：

```
package p
public class Pair<T,U> { . . . }
```

若需在 Objective-C 代码中操作 Pair<Int, Bool> 实例，需在 cjc 镜像生成配置文件的 [[package]] 条目中为 p 添加 generic_object_configuration 属性，内容如下：

```
#      -    -    -
[[package]]
name = "p" generic_object_configuration = [
        { name = "Pair", type_arguments = [ "Int, Bool" ] },
#               -    -    -
```

cjc 随后将生成一个非泛型的 Objective-C 类：

```
@interface PairIntBool
     -    -    -
@end
```

代表该特定类型的*实例化*：Pair<T,U>。 更多详情请参阅泛型实例化。

仓颉码到Objective-C映射

当前版本的cjc编译器采用本章所述的仓颉→Objective-C映射方案。适用命令行选项请参阅《仓颉镜像生成参考》。

通用注意事项

仓颉的类型系统与Objective-C语言存在差异。某些仓颉类型及其特性在Objective-C中既无直接对应，也无合理近似等价物，因此当前版本对其支持有限，部分类型甚至完全不支持。

因此，若公共成员、函数/构造函数参数或函数返回值的类型不受支持，则对应成员、函数或构造函数将无法镜像。当前编译器不会将此类用法标记为错误，仅直接忽略镜像操作。

镜像类型应编译为独立动态库，并禁用ARC支持（使用Objective-C编译器选项-fno-objc-arc）。详情请参阅[类与接口(#cangjie-classes-and-interfaces)]章节的说明。

名称

当前保留了Cangjie包、函数、类型及类型成员的原始名称，这意味着其中部分名称可能与Objective-C关键字（如"int"）冲突，或包含Objective-C命名规则中不允许的字符。

布尔与数值类型

在Objective-C类型中存在对应的仓颉布尔值和数值类型将映射至对应类型；不存在对应类型（如Float16）的则不予支持：

仓颉类型	Objective-C类型
Bool	BOOL
Int8	int8_t
Int16	int16_t
Int32	int32_t
int64	int64_t
Int	int64_tI
ntNativessize_tUInt8	uint8_t
uint16	uint16_t
uint32_t	uint32_t
uint64_t	uint64_t
UInt	uint64_tUIntNati
ve	size_tFloat16
	不支持
Float32	float
Float64	double

有关不支持类型的处理信息，请参阅一般注意事项。

Rune

Rune 类型不受支持。

有关未支持类型的处理信息，请参阅通用注意事项。

特殊类型

类型 Unit 仅作为函数返回类型受支持，映射至 Objective-C 的 void。

类型 Nothing 无法被支持。类型 Any 目前尚未

支持。

有关未支持类型的处理方式，请参阅通用注意事项。

元组

元组类型目前尚未支持。

有关未支持类型的处理信息，请参阅一般注意事项。

结构体类型

公共仓颉结构体类型定义被映射到带有 objc_subclassing_restricted 属性的 Objective-C 类定义中。这些定义由 cjc 编译器自动生成，包含在 Objective-C 和仓颉之间传输控制和数据的粘合代码。请勿手动修改这些定义。

```
package cj

import interoplib.objc.*

public struct Vector {let x:
    Int32
    let y: Int32

    public init(x: Int32, y: Int32) { this.x = x
        this.y = y
    }

    public func add(v: Vector): Vector { Vector(x + v.x,
        y + v.y)
    }
}
```

```
// Vector.h
#import    <Foundation/Foundation.h>
#import    <stddef.h>
__属性__ ((objc_subclassing_restricted))
@interface Vector : NSObject
// 辅助粘合代码方法
- (id)init:(int32_t)x :(int32_t)y;
- (Vector*)add:(Vector*)v;
```

```
// 更多辅助粘合代码方法
@end
// Vector.m

// 粘合代码

@implementation Vector

// 粘合代码

- (id)init:(int32_t)x :(int32_t)y {
    // 胶水代码创建仓颉向量实例(x, y)并
    // 将其关联至'self'。
}
- (Vector*)add:(Vector*)v {
    // 粘合代码，获取关联于'self'和'v'的仓颉向量实例
    // 关联'self'和'v'的Cangjie Vector实例，调用add()成员函数
    // 并传入参数 Cangjie-v，随后
    // 创建新的Vector实例并关联至
    // add() 调用的结果。
}

// 粘合代码

@end
```

仅镜像公共结构体成员和通用构造函数。

成员函数映射为方法，其中参数类型和返回值类型替换为对应的镜像类型。返回Unit的成员函数映射为void方法。实例成员函数映射为实例方法（前缀为减号-），静态成员函数映射为类方法（前缀为加号+）。

支持类型的**成员属性**将映射为同名的@property声明，其类型映射自仓颉属性类型并标注readonly属性。获取器映射为无参数方法，名称与属性相同，返回类型为仓颉属性类型的映射值。**注意：**当前*暂*不映射设置器，因此可变成员属性的映射结果同样为只读属性。

支持类型的**成员变量**会被镜像为同名的@property声明，其类型对应于仓颉变量类型，并标记为只读属性。系统会合成获取器以实现访问功能。**注意：**当前不会为var成员变量合成设置器，因此后者的镜像同样为只读属性。

构造函数将映射为参数类型替换为对应镜像类型的构造函数，*包括可能隐式声明的默认构造函数。*

当前版本对可映射的结构体类型施加了诸多严格限制，甚至可以说结构体类型需专门为暴露给苍颉而设计：

- 不支持函数重载：重载成员函数的镜像将生成相同名称，导致名称冲突。
- 当前版本中，let与var成员变量均映射为只读属性，因此后者无法通过Objective-C代码修改。此限制将在未来版本中解除。过渡期间可通过添加setter函数实现替代方案。
- 可变实例成员函数的镜像功能尚在开发中。虽然已实现镜像，但在某些情况下生成的胶合代码存在错误。
- 映射后的结构体可实现除Any之外的接口，但"im-plements"关系*不会*传递至Objective-C：生成的@interface指令不会在类名后以尖括号<>标注所实现接口的映射名称。
- 成员运算符函数的镜像化不被支持。在镜像生成过程中，它们会被跳过。
- 泛型结构体通过单态化实现镜像支持。详情参见泛型说明。
- 具有不受支持参数类型的结构体成员函数和构造函数，以及具有不受支持返回类型的成员函数，均不会被镜像。
- 镜像结构体可能包含直接扩展和/或接口扩展，但在镜像生成过程中这些扩展将被忽略。
- 成员函数不得包含任何类型参数。

类与接口

仓颉类和接口定义分别映射为 Objective-C 类和协议定义。这些定义包含自动生成的粘合代码，不得进行任何修改。

```
package cj

import interoplib.objc.*

public interface Valuable { public func
    value(): Int
```

```
}

public open class Singleton <: Valuable { private let
    _v: Int
    public init (v: Int) { _v = v } public func
    value(): Int { _v }
}


public class Zero <: 单例 {   public
    init() {      super(0)   }
}
```

```
// Valuable.h
@protocol Valuable
- (int64_t)value;
@end

// 单例.h
// 粘合代码
@interface 单例 : NSObject
// 粘合代码
-(id)init:(int64_t)v;
- (int64_t)value;
// 粘合代码
@end

// Singleton.m
// 粘合代码
@implementation Singleton
// 粘合代码
-(id)init:(int64_t)v   {
    if (self =[super init]) {
        // 胶水代码创建仓颉单例实例(v)
        // 并将其关联到 `self`。
    }
    return self;
}
- (int64_t)value {
    // 胶水代码调用仓颉库的value()成员函数
    // 关联'self'的仓颉单例实例并返回结果
}
// 粘合代码
@end

// Zero.h
#import   "Singleton.h"
__属性__ ((objc_subclassing_restricted))
@interface Zero : Singleton
```

```
// 粘合代码
- (id)init;
// 粘合代码
@end

// Zero.m
// 粘合代码
@implementation Zero
- (id)init   {
    if (self =[super init]) {
        // 胶水代码创建仓颉零()的实例
        // 并将其关联到 `self`。
    }
    return self;
}
@end
```

当前，即使标识符与Objective-C关键字（如int）冲突，只要其作为镜像类型及其成员的名称，仍会被保留。

非公开类的镜像会标记为objc_subclassing_restricted属性。

公共成员和构造函数会被镜像。此外，开放类的受保护开放实例成员函数也会被镜像，以便子类能够重写它们。其他类或接口成员均不进行镜像。

> **注意：**由于Objective-C中既没有包也没有其他类型的命名空间，因此此类受保护的公开成员函数的镜像在Objective-C代码的任何位置均可访问。

当前版本中**成员变量**不会被镜像，且不提供从Objective-C访问它们的途径。

当前版本中**成员属性**不进行镜像处理。

成员函数被映射为方法，其中参数类型和返回值类型被替换为相应的镜像类型。返回 Unit 的成员函数被映射为 void 方法。静态成员函数被映射为类方法（前缀为“+”），实例成员函数被映射为实例方法（前缀为“-”）。

普通构造函数映射为构造方法，其中参数类型替换为对应的映射类型。*这包括可能隐式声明的默认构造函数。*

重要提示：从外部看，映射Canglie类和接口的类型在所有方面都是标准的Objective-C类和协议：它们可被继承/实现，其方法可通过常规Objective-C方法重载和覆盖，实例类型可被查询等等。然而，两种语言的差异会引发若干潜在问题，在Objective-C代码中使用此类类型时需格外谨慎：

1. Objective-C 方法不存在禁止覆盖的属性，因此映射开放类中非公开成员函数的方法虽可被覆盖（但显然不应如此操作）。

2. 当前实现不支持覆盖开放类中静态仓颉方法的镜像。这些方法被映射为Objective-C类方法（以加号+为前缀），子类中绝不应尝试覆盖。编译时无法强制执行此限制。

3. 在Cangjie中构造函数不会被继承，而Objective-C中的初始化方法则与其他方法一样会被继承，这可能导致不良后果。例如，考虑以下Cangjie类：

   ```
   public class A {
       public init() {...}
   }
   public class B <: A { private
       init() {...}
       public init(x: Int) {...}
   }
   ```

   在仓颉类中，B类不会暴露其无参的公共构造函数。然而，B类的镜像会从A类的镜像继承无参的init方法，因此表达式[[B alloc] init]能够编译并运行，但从A类继承的init方法会创建并关联一个仓颉类A的实例，而非B类实例。

注意：开源仓颉类别的镜像功能依赖于低级别的Objective-C特性，这些特性与自动引用计数（ARC）不兼容。因此所有镜像类型通常应编译为独立的动态库，并禁用ARC支持（使用Objective-C编译器选项-fno-objc-arc）。

当前版本对可镜像的类与接口类型施加了诸多严格限制，甚至可以说此类类型需专门为暴露给仓颉而设计：

- 抽象类不会被镜像，且实现抽象类的具体类的镜像无法编译。
- 不支持函数重载：重载成员函数的镜像将生成相同名称，导致名称冲突。
- 成员变量不予镜像，且不提供访问机制。此限制将在未来版本中解除。当前可通过手动为仓颉类添加获取/设置函数作为临时解决方案。
- 目前不支持镜像实现除 Any 之外接口的类，因为在镜像生成过程中会丢失显式接口实现信息。

- 泛型类的镜像功能通过单态化实现。更多信息请参阅泛型部分。
- 包含公共成员属性、成员运算符函数或基本构造函数的类/接口无法进行镜像处理。此类实体将直接跳过镜像操作且不触发警告。
- 对于成员函数和构造函数，唯一普遍支持的参数类型为布尔型和数值类型。 对于成员函数，唯一普遍支持的返回值类型为Bool、数值类型和Unit。特别针对非公开类的成员函数和构造函数，若结构体、枚举类型及非公开类定义于同一包内，则可作为参数和返回值类型使用。参数类型或返回类型不符合规范的成员函数与构造函数均不会被镜像。
- 镜像生成过程中不会保留接口之间的继承关系。
- 镜像生成过程中不会将接口实现信息传播至Objective-C（注意上例中Singleton.h文件的@interface指令末尾未包含<Valuable>）。
- 镜像类可能包含直接扩展和/或接口扩展，但在镜像生成过程中这些扩展将被忽略。

枚举

仓颉枚举被镜像为一个具有 objc_subclassing_restricted 属性的 Objective-C 类，该类具有与枚举构造函数（工厂方法）匹配的静态方法，且没有公共构造函数。

```
public enum TimeUnit {
    | Year(Int)
    | Month(Int)
    | 年
    | 月
}

__属性__ ((objc_subclassing_restricted))
@interface TimeUnit : NSObject
// 粘合代码
+ (TimeUnit*)Year:(int64_t)p1;
+ (TimeUnit*)Month:(int64_t)p1;
+ (TimeUnit*)Year;
+(TimeUnit*)Month;
// 粘合代码
@end
```

枚举构造函数会被映射为名称相同、参数类型和数量匹配的静态工厂方法；参数名称将被合成（p1, p2, ...）。

仅镜像公共枚举成员。

成员函数映射为方法，其中参数类型和返回值类型替换为对应的镜像类型。返回Unit的成员函数映射为void方法。静态成员函数映射为类方法（前缀为“+”），实例成员函数映射为实例方法（前缀为“-”）。

支持类型的**成员属性**将映射为同名的@property声明，其类型映射自仓颉属性类型并标记为只读。获取器映射为无参数方法，名称与属性相同，返回类型为仓颉属性类型的映射值。**注意：**当前不映射设置器，因此可变成员属性的映射结果同样为只读。

当前版本对可映射枚举施加了若干限制：

- 不支持函数重载：重载成员函数的镜像会以相同名称生成，导致名称冲突。
- 镜像枚举可实现除 Any 之外的接口，但“实现”关系*不会*传播至 Objective-C：生成的 @interface 指令不会在类名后以尖括号 <> 标注所实现接口的镜像名称。
- 泛型枚举的镜像功能尚未支持；将通过单态化实现支持。详见泛型说明。
- 对于成员函数和构造函数，仅支持布尔型和数值型参数类型。成员函数的返回值类型仅普遍支持布尔型、数值型及Unit类型。参数类型或返回类型不符合规范的成员函数与构造函数均不会被镜像。
- 包含成员运算符函数的枚举目前不支持镜像。尝试镜像此类枚举将导致生成无效的Objective-C代码。
- 参数类型不受支持的枚举成员函数和构造函数，以及返回类型不受支持的成员函数均不会被镜像，且不会触发警告。
- 镜像的枚举可能包含直接扩展和/或接口扩展，但在镜像生成过程中这些扩展将被忽略。

递归定义的枚举受支持：

```
public enum Peano {
    | Z
    | S(Peano)

    public func toInt(): Int { match
        (this) {
                                case Z => 0
            case S(x) => 1 + x.toInt()
        }
    }
}
```

泛型

Objective-C与仓颉泛型的本质存在差异，因此后者无法映射至前者。然而，仓颉类型参数化的*具体实例*（如G<Int64>）属于具体类型，故可映射为非泛型的Objective-C类型。此类类型亦称为*单态化泛型*。

注意：当前版本仅支持将使用基本类型作为类型参数实例化的泛型类型进行映射。具有自身类型参数的成员函数同样不支持映射。

请通过cjc编译器的配置文件指定需映射的类型实例化形式。详情参见《泛型实例化规范》。

示例：

编译以下仓颉类定义时：

```
public class G<T> { private let
    _t: T
    public init(t: T) { _t = t }public
    func get(): T { _t }
}
```

在配置文件的相应部分添加以下内容：

```
    - - -
通用对象配置                                = [
    { name = "G", type_arguments = ["Bool", "Int"] },
]
    - - -
```

编译器将为 G<Bool> 和 G<Int> 生成 Objective-C 镜像类，分别命名为 GBool 和 GInt。

完整说明请参阅《仓颉镜像生成参考》章节。

空值处理

由于仓颉没有空值类型，若将nil作为参数传递给仓颉镜像方法，将在胶合代码中引发运行时异常：

```
public class Node {
    private let next: ?Node public
    init() {
        next = None
    }
    public init(next: Node) {                    /* 纯仓颉 */ this.next =
        Some(next)
    }
}
```

```
__属性__ ((objc_subclassing_restricted))
@interface Node
/* 胶水代码 */
- (id)init;
- (id)init:(Node *)next;
/* 粘合代码 */
@end
```

```
    Node *list = [[Node alloc] init:nil];                    /* 空指针异常 */
```

反之，目前尚无法从兼容 Objective-C 的类型公共成员函数中返回 nil。

注意：当前版本支持自动处理 Option<T> 的（解）封装，其中 nil 映射为 None，但仅限于 Objective-C 类指针和协议指针映射至苍楞的情况，反向映射不适用。详见 nil 处理说明。

仓颉映射生成参考

在仓颉SDK中，不存在独立的镜像生成器用于将仓颉字型暴露给Java和Objective-C代码。cjc编译器本身在编译仓颉字型的源代码时，即可为其生成对应的Java/Objective-C镜像字型定义。

命令行语法

以下附加cjc选项可在编译过程中启用并控制仓颉类型的镜像生成：

--experimental *(必选)*

当前版本必须指定此选项，因跨平台功能尚在开发中，但未来将不再需要。

--enable-interop-cjmapping=Java *或*
--enable-interop-cjmapping=ObjC *(必填)*

分别启用生成Cangjie字符集的镜像类型定义，格式为Java或Objective-C源代码文件。

--output-interop-cjmapping-dir *路径名* *(可选)*

*路径名*必须指向一个目录，cjc编译器将在该目录中放置其生成的包含镜像类型定义的Java或Objective-C源代码文件。若*路径名*未指向现有文件系统位置，cjc将尝试在此处创建目录。若*路径名*指向非目录结构，cjc将终止并输出错误信息。

*路径名的*默认值为./java-gen或./objc-gen，具体取决于--enable-interop-cjmapping参数分别设置为“Java”或“ObjC”。

--import-interop-cj-package-config-path *路径名* *（可选）*

*路径*名必须指向定义哪些仓颉类型作为镜像类型暴露给 Java 或 Objective-C 的配置文件。详情请参阅仓颉镜像生成配置。

仓颉镜像生成配置

仓颉镜像生成配置文件采用TOML语法的纯文本格式，其中指定：

- 非通用仓颉类型需映射
- 泛型仓颉类型的具体实例需镜像

默认值

[default] 表定义了以下包的默认设置：要么完全未提供包特定设置，要么这些设置不会覆盖默认值。

属性：

API策略 *（可选）*

字符串，用于定义是否默认镜像所有公共实体。有效值：

- "Full" - 默认镜像所有公共实体
- "None" - 默认不镜像任何公共实体

通用类型策略 *(可选)*

定义泛型公共实体是否默认进行镜像的字符串。有效值：

- "部分" - 仅镜像generic_object_configuration中显式列出的实例化对象
- "None" - 默认不镜像任何泛型公共实体

包

[[packages]]数组中的每个条目指定一个独立的Cangjie源包名称，并可选包含：

- 一个过滤器，用于定义该包中哪些类型应包含在镜像集内或排除在镜像集外
- API镜像策略（若非默认值）
- 通用类型镜像策略（若非默认值）
- 需镜像的特定通用类型实例化

属性：

名称 *(必填)*

包含*源*仓库名称的字符串，本[[packages]]数组条目中的设置将应用于该仓库。

示例：

```
name = "com.example.VectorMath"
```

API策略 *(可选)*

字符串，用于定义默认是否镜像指定包的所有公共实体。有效值：

- "全量" - 所有公共实体默认进行镜像
- "None" - 默认不镜像任何公共实体。若此属性缺失，则使用默认值。included_apis *(可选)*

字符串数组，每个元素包含需镜像的公共实体名称。若数组包含类型成员的限定名称，则该类型本身也将被镜像。在给定的 [[packages]] 数组条目中，included_apis 或 excluded_apis 仅可存在其中一项，不可同时存在。

示例：

```
included_apis = [
    "Vector.product", // Vector 类型也将暴露
    "HiddenV"
]
```

excluded_apis *(可选)*

一个字符串数组，每个字符串包含给定包中定义的、*不应*镜像的公共仓颉实体名称。在给定的[[packages]]数组条目中，可以包含`included_apis`或`excluded_apis`，但不能同时包含两者。

示例：

```
excluded_apis = [
    "Vector.product", // 即使暴露了Vector，product也不被镜像
    "HiddenV"
]
```

通用类型策略 *(可选)*

定义本包中定义的泛型公共实体是否默认进行镜像的字符串。有效值：

- "Partial" - 仅镜像generic_object_configuration中显式列出的实例化对象
- "None" - 默认不镜像任何泛型公共实体若该属性缺失，则使用默认值。泛型

对象配置 *(可选)*

定义需镜像的泛型实体具体实例化的表列表。详情参见《泛型实例化》。

泛型实例化 [[packages]]数组条目的 generic_object_configuration 属性定义了将镜像哪些特定的通用仓颉实体实例。

该属性的值是一个（可能为空）的表数组。数组中的每个条目都必须包含以下两个属性：

name

包含当前包中定义的公共通用仓颉类型或全局函数名称的字符串。

type_arguments

字符串数组，每个元素包含名称属性中指定的泛型类型或全局函数的有效类型参数。

简言之，"有效"意味着该类型/函数可在尖括号< >内使用给定字符串进行实例化。

考虑以下仓颉类：

```
public class G<T> {
    public func f(t: T): Unit {}
```

```
        public func f(b: Bool): Unit {}
    }
```

G<Bool> 无法实例化，因此泛型对象配置

```
                                                = [
        { name = "G", type_arguments = ["Bool"] }
    ]
```

将触发编译器错误。

注意：当前版本仅支持使用基本类型作为类型参数实例化的泛型类型镜像。

示例：

假设存在以下仓颉定义public class G<T> { . . .

```
}
public func f<T>(): Unit { . . . } public
struct S<T,U> { . . . }
```

属性

```
generic_object_configuration                = [
        { name = "G", type_arguments = ["Int32"] },
        { name = "f", type_arguments = ["Float32", "Float64"] }
        { name = "S", type_arguments = ["整数, 布尔值"] }
]
```

指示cjc编译器生成以下泛型实例化及其镜像：

```
        G<Int32>
        f<Float32>
        f<Float64> S<Int,
        Bool>
```

镜像分别命名为 GInt32、fFloat32、fFloat64 和 SIntBool。

在Cangjie中使用Objective-C

在首个互操作场景（详见《在Objective-C中使用仓颉》）中，用户定义的仓颉类型被映射为Objective-C类型，其行为完全符合常规Objective-C类与协议规范：可自由扩展类、实现协议、创建实例并传递等。

iOS版仓颉SDK同样支持反向映射功能，可将Objective-C类与协议映射为仓颉类与接口，并支持扩展、实现及实例化操作。但具体功能集、限制条件及工具链存在差异：镜像类型与

函数声明需通过独立工具生成，而非由cjc编译器完成。

因此启用此第二种场景的过程更为复杂。具体步骤如下：

1. 基于Objective-C类与方法设计互操作层

   开发者 → 互操作层设计（Objective-C*伪代码*）

2. 为互操作层设计中使用的所有Objective-C类和协议生成镜像声明。

   .h → 镜像生成器 → .cj（镜像文件）

3. *使用仓颉*编写构成互操作层的类。按需使用镜像的Objective-C类型：创建实例、调用成员函数等。

   互操作层设计 + .cj（镜像）→ 开发者 → .cj（互操作层）

4. 编译互操作性层类（简称*互操作类*）和镜像类型，同时使用cjc工具。cjc将生成：

   - 所有镜像类型使用场景所需的粘合代码。
   - 互操作性层中Objective-C部分的实际Objective-C源代码（即互操作类的*Objective-C封装器*）

   .cj（镜像类型 + 互操作层）→ cjc → .dylib + .h/.m（互操作层）

5. 添加至您的 iOS 项目：

   - cjc生成的.h和.m文件
   - 由cjc生成的.dylib文件
   - 来自仓颉 SDK 的运行时库 (.dylib)

   iOS项目 + .h, .m, .dylib → iOS工具链 → iOS应用程序

6. 现在您可以在应用程序的Objective-C代码中开始使用互操作类封装器，从而实现从Objective-C调用仓颉代码的功能。

重要提示：当前版本的iOS版仓颉SDK还需从仓颉开源库下载Cangjie.h头文件并集成到XCode项目中：

https://gitcode.com/Cangjie/cangjie_runtime/blob/dev/runtime/src/Cangjie.h

初始互操作类创建流程

环境配置

1. 安装 llvm@16 Homebrew 公式：

   brew install llvm@16

2. 将 llvm@16/lib/ 子目录添加至 DYLD_LIBRARY_PATH 环境变量：

   export DYLD_LIBRARY_PATH=/opt/homebrew/opt/llvm@16/lib:$DYLD_LIBRARY_PATH

3. 运行仓颉 SDK 中的 envsetup.sh 脚本。

4. 通过在终端执行以下命令验证安装：

   ObjCInteropGen

   若开始出现镜像生成器使用说明，则您已具备使用 仓颉SDK的iOS互操作功能所需的一切条件。

步骤1：设计互操作层

在此步骤中，您*需从Objective-C代码视角设计*一个或多个互操作类的API，即针对每个互操作类需确定：

- 该类将继承自哪个Objective-C基类（如NSObject）；
- 需实现哪些Objective-C协议（如有）；
- 该类将包含哪些公共/受保护的方法和/或构造函数。您只需了解参数类型，以及方法的名称和返回值类型；具体实现将在Cangjie中编写。

另请参阅《互操作类的特性与限制》。

对于应用程序的Objective-C部分，互操作性层与常规Objective-C类集无法区分。因此，您可以采用Objective-C伪代码的形式进行设计。编写一个或多个Objective-C类的规范说明及@interface定义。 **注意：**您无需编写对应的@implementation部分，因为后续步骤中您仍需用Cangjie重新编写这些内容。

支持的参数类型：任何映射的Objective-C类型。

支持的返回类型：任何映射的Objective-C类型或void。

支持的继承关系：互操作类只能继承映射的Objective-C类（*不可继承*其他互操作类！），且只能实现映射Objective-C协议的接口。

限制：

- 不支持可变参数（varargs）方法和构造函数。

- 互操作类及其非私有成员函数均不可为泛型类型。
- 泛型Objective-C类型在镜像生成前会被擦除；所有类型变量将替换为对应的上界。

端到端示例：

为简化起见，假设您的iOS应用程序中已存在一个名为M的类，该类包含一个无参数的空实例方法foo()：

```
// M.h
#import    <Foundation/Foundation.h>

@interface M : NSObject
- (void)foo;
@end

// M.m
#import "M.h"

@implementation M
- (void) foo {
      printf("Hello from ObjC M.foo()\n");
}
@end
```

你希望在M类的子类A中，用仓颉字法实现覆盖该foo()方法。

此时互操作类的设计应如下所示：

```
#import "M.h"@interface A
: M
- (void)foo;
@end
```

步骤 2：生成镜像类型声明

现在，您需要为互操作类所依赖的所有 Objective-C 类型生成镜像类型声明：包括其父类型、成员属性与局部变量的类型、方法参数/返回值的类型，以及若上述类型中存在数组类型时，其元素的类型。

继续操作前，请先按常规方式构建您的 iOS 应用程序（无需修改任何内容），以确保镜像生成器接收完整且一致的 Objective-C 头文件集作为输入。

随后编写相应的镜像生成器配置文件并运行镜像生成器：

ObjCInteropGen --mode=normal <config-file>

其中<config-file>为配置文件的路径名。

端到端示例（续）：

类A的唯一直接依赖关系是其父类M，因此配置文件非常简单：

```
# A.toml
# 将M及其所有依赖项的镜像放置于'cjworld'包中： [[packages]]
filters = { include = ["M", "NS.+"] } package-
name = "cjworld"

# 将具有镜像类型定义的输出文件写入当前目录：[输出根目录.默认]
路径 = "."

# 指定输入头文 件 的 路径名：
[sources.all] paths =
["M.h"]

[sources-mixins.default]
sources = [".*"] arguments-
append = [
    # 若出现"未知类型名称"错误，请取消注释以下行 # "-DTARGET_OS_IPHONE=1",

    # 请根据您系统中 Objective-C 头文件的位置修改以下路径名"-F",
    "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/System/Library/Frameworks","-isystem",
    "/Library/Developer/CommandLineTools/usr/lib/clang/17/include",
    "-isystem",    "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include"
]
```

镜像生成器命令行：

ObjCInteropGen --mode=normal A.toml

上述命令将生成文件 cjworld/M.cj，其中包含类 M 的镜像类型声明，以及一系列 cjworld/NS*.cj 文件，这些文件包含 M 依赖的 Foundation 框架类和协议的镜像类型声明。

常见配置问题镜像生成器无法找到标准头文件，例如`stdarg.h`或`stdbool.h`。

示例错误信息：

```
..../CoreFoundation.h:19:10: error: 'stdarg.h' file not found
```

这通常意味着[sources-mixins]表中arguments-append数组的路径名不正确。请仔细核对它们是否与系统中头文件的位置相符。

镜像生成器报告“未知类型名称‘NSUInteger’”错误或类似错误。

示例错误信息：

.../NSObjCRuntime.h:626:74: error: 未知类型名称 'NSUInteger'

在某些系统上，需要向Clang传递额外参数"-DTARGET_OS_IPHONE=1"或"-DTARGET_OS_OSX=1"。

将其添加至[sources-mixins]表的arguments-append数组中。在上例中该参数已存在但被注释掉：

```
    -    -    -
arguments-append = [
     # 若遇到"未知类型名称"错误，请取消注释以下行 # "-DTARGET_OS_IPHONE=1",
    -    -    -
```

步骤 3：编写互操作类

针对步骤 1 中互操作层设计中包含的每个 Objective-C 类骨架，按如下方式编写对应的仓颉类：

- 使用适当的包名和类名（cjc生成的Objective-C封装类将具有相同的完全限定名）。
- 导入 interoplib.objc.*。
- 若步骤二中生成了镜像类型，则导入这些类型。切勿导入所有生成的依赖项，仅导入实际需要的类型。
- 为互操作类添加 @ObjCImpl 注解。
- 让互操作类继承相应的开放镜像类，或者在设计中没有显式父类时继承NSObject。
- 使用注解标记公共类成员和构造函数：`@ForeignName["*foreign-name*"]`其中`*foreign-name*`即该成员或构造函数所需的Objective-C名称
  构造函数的期望Objective-C名称，遵循以下规则：

  1. *请勿*为重写镜像超类成员函数的成员函数添加注释。若互操作类的此类成员函数存在 @ForeignName 注释，将触发编译时错误。

     此类覆盖函数的Objective-C名称必须与被覆盖的Objective-C方法名称完全一致。

后者通过镜像类型声明中的@ForeignName注解自动从Objective-C传播至Cangjie（详见《类与协议》章节）。cjc编译器会捕获这些注解。

2. *请*为具有*两个或更多*参数的构造函数和非重写成员函数添加注解。若此类成员函数或构造函数缺少 @ForeignName 注解，将引发编译时错误。

   编译器无法自动将具有两个或更多参数的成员函数或构造函数的原始仓颉名称转换为有效的Objective-C名称。此类名称本质上由相应数量的片段组成，各片段以冒号:分隔。

3. 重载的成员函数和构造*函数必须*使用@ForeignName标注赋予独特的Objective-C名称，因为Objective-C不支持方法重载。其中仅允许最多一个参数的重载实体保留原始仓颉名称。

4. 其余成员与构造函数是否添加@ForeignName注解可选。若未添加注解，则该成员的原始仓颉名称将作为对应实体的Objective-C名称使用，并以冒号分隔。

   附加于单参数方法名称之后，而与唯一公共构造函数（带一个或零个参数）匹配的初始化方法则分别称为 init: 或 init。

注意：当前版本的cjc编译器*尚未*完全验证*外部名称*。特别是，它不会检查冒号分隔符的数量是否与方法或构造函数参数数量匹配。
与方法或构造函数参数数量是否匹配。

- 使用以下Objective-C到仓颉的类型映射 （T'为匹配的值类型或对应镜像类型）：

Objective-C	Cangjie	备注
void	单元	
BOOL	布尔	
有符号字符	Int8	
短整型	Int16	
int	Int32	
long	Int64	
long long	Int64	
unsigned char	UInt8	
无符号短整型	UInt16	
unsigned int	UInt32	

Objective-C	仓颉	备注
unsigned long	UInt64无	
符 号 长 长 整 型 UInt64 浮 点 型		
	Float32	
双精度	浮点64	
struct	@C struct'	(*)
枚举 w/基类型 T	T'	
id	对象ID	(†)

Objective-C	仓颉	如果 T 是...	备注
T*	CPointer<T'>...	基本类型或结构体类型	
T*	CPointer<U'>...	枚举类型，且 U 是其基类型	
T*	CFunc<T>	. . . 纯C函数类型	
T*	T'	. . . 类类型	(†)

(*) Objective-C结构体不得包含非CType兼容类型的字段。详情参见结构体章节。

(†) 对于可能接收/持有 Objective-C nil 值的镜像类型和互操作类类型，请使用 ?<T'> (Option<T'>) 类型作为参数、返回值和局部变量。

详见《Objective-C 到仓颉映射》。

支持特性：

互操作类的成员函数：

- 可重写/重新定义其镜像超类的公开实例/静态成员函数，实现其镜像超接口的成员函数，并可引入新成员函数。

互操作类的构造函数与成员函数：

- 在仅处理常规仓颉类型（及其值）时，主体可使用任何仓颉语言特性。
- 可使用常规仓颉语法：
  - 实例化互操作类与镜像Objective-C类的对象
  - 调用互操作类和镜像的Objective-C类型的静态方法与实例方法（包括使用super调用Objective-C的超构造函数和超类方法）
  - 访问自身成员属性及其他互操作类和镜像Objective-C类型的非私有成员属性

限制：

- 互操作类可实现映射的Objective-C协议接口，但不可实现常规的Cangjie接口。
- 互操作类不得声明为公开或抽象类，且不可通过extend进行扩展。
- 互操作类的非私有成员函数与构造函数：
  - 参数与返回值类型仅限使用上述列出的Objective-C映射类型。（因此不支持可变长度参数，因其需使用无Objective-C映射的Cangjie Array<T>类型）
  - 不得使用命名参数。
  - 不得包含类型参数（即不得为泛型）。
- 非私有成员属性仅可采用上述列出的Objective-C映射类型。
- 泛型Objective-C类型将被擦除；镜像类型将用上界替代对应的类型参数。
- **重要提示：**镜像类型和互操作类的实例（即Objective-C引用类型的值）不得逃逸到全局或静态的仓颉变量中，也不得逃逸到任何在调用之间持久存在的数据结构中。

端到端示例（续）：

延续上例，互操作类 A 的实现如下：

```
package cjworld                          // 相同包名

import interoplib.objc.*    // 必须始终包含

@ObjCImpl
public class A <: M {

    public init() { super()
    }

    @ForeignName["foo"]
    override public open func foo(): Unit { println("Hello from
        overridden A.foo()")
    }

}
```

步骤 4：编译互操作类

命令行：

```
cjc --target=arm64-apple-ios-simulator \
    --sysroot=$(xcrun --show-sdk-path --sdk iphonesimulator) \
    --输出类型=dylib \
    --整数溢出=包裹 \
    <源文件> \
    -o <目标文件> \
    --link-options "-undefined dynamic_lookup"
```

其中

<source-files> 是互操作类和镜像类型声明的源代码文件。

<目标文件> 是输出 .dylib 文件的期望名称，其中包含互操作类的编译后仓颉代码，例如 libcjworld.dylib。

编译器还将为Objective-C封装器或互操作类生成Objective-C源文件（.h和.m），并将其放置在./objc-gen/子目录中。

该 .dylib 文件必须进行签名：

```
xcrun codesign --sign - <dylib-file>
```

端到端示例（续）：

编译互操作类：

```
cd cjworld

cjc --target=arm64-apple-ios-simulator \
    --sysroot=$(xcrun --show-sdk-path --sdk iphonesimulator) \
    --输出类型=dylib \
    --int-overflow=wrapping \
    *.cj \
    -o libcjworld.dylib \
    --link-options "-undefined dynamic_lookup"
```

编译器将生成三个文件：./libcjworld.dylib、./objc-gen/A.h 和 ./objc-gen/A.m。

对生成的动态库进行签名：

```
xcrun codesign --sign - libcjworld.dylib
```

步骤5：整合所有内容

- 在项目中创建另一个子目录，并将 $CANGJIE_HOME/runtime/lib/ios_simulator_aarch64_cjnative/

目录中的所有动态库复制到该子目录中。

- 将所有动态库文件*以及*上一步生成的.dylib文件作为依赖项添加到Xcode项目中（在“BuildPhases”下的“Copy Files”和“Link Binary With Libraries”列表中均添加）。
- 将上一步生成的.h和.m文件移至项目根目录，然后重新构建项目。

端到端示例（续）：

在项目根目录创建新子目录，并将构成iOS版仓颉运行时的动态库复制至该目录：

```
cd ..
mkdir -p CJRuntimeDylibs
cp $CANGJIE_HOME/runtime/lib/ios_simulator_aarch64_cjnative/*.dylib CJRuntimeDylibs/
```

将这些动态*库及* ./cjworld/libcjworld.dylib 添加至 Xcode 项目依赖项（在“BuildPhases”下的“Copy Files”和“Link Binary With Libraries”列表中均添加）。

将cjc生成的.h和.m文件移至项目根目录：

```
mv cjworld/objc-gen/*.h ./mv
cjworld/objc-gen/*.m ./
```

重新构建Xcode项目。

从仓颉调用Objective-C

完成前文所述的互操作层设计、构建与集成后，即可在互操作类的成员函数中添加使用Objective-C类型的代码。类型映射规则保持不变：

通过互操作类封装器的相应方法，在Objective-C中添加调用仓颉函数的代码，并重新构建iOS项目。

端到端示例（续）：

在Objective-C代码中插入A类的实例化操作及该实例的foo()方法调用：

```
#import "M.h"
#import "A.h"

M* a = [[A alloc] init]; [a foo];
```

重新构建 Xcode 项目，并在 iOS 模拟器上运行应用程序。

后续增强

通过上述流程，您应已掌握如何向互操作层添加更多类，以及如何在仓颉编码中启用更多 Objective-C 类。以下为简要总结及各子章节链接：

步骤1：按需向设计中添加更多互操作类，并/或通过新增方法和/或属性增强现有类。

步骤2：若涉及新增的未映射Objective-C引用类型，请先构建*当前*版本应用以确保一致性，随后调整镜像生成器配置文件，并重新生成所有镜像类型声明。

步骤3：根据需要使用镜像生成的Objective-C类型，实现新增的互操作类（如有）并/或修改现有类的代码。

步骤 4：将所有互操作类合并编译。

步骤5：将cjc在上一步生成的.h、.m和.dylib文件复制到项目中的相应位置，然后重新构建项目。

现在您可以在应用程序的 Objective-C 部分开始使用新添加的互操作类和/或方法。

互操作类的特性与限制

1. 互操作类*必须*是镜像类的直接子类。
2. 互操作类*可以*实现一个或多个镜像接口，但绝不能实现常规的仓颉接口。反之，常规的仓颉类型不得实现或继承映射Objective-C协议的接口。
3. 互操作类不得声明为公开或抽象类，且不可通过extend进行扩展。
4. 互操作类可引入*任意仓颉类型*的新实例字段，并重写其镜像父类的成员函数。
5. 互操作类的构造函数可通过super()调用父类构造函数，但实例成员函数调用受限于常规Cangjie构造函数的约束，且必须初始化所有新增成员变量。
6. 互操作类的实例成员函数可调用该类从镜像父类继承的实例成员函数，并/或使用 super() 调用该类覆盖的父类成员函数。

7. 所有镜像类型和互操作类的构造函数及成员函数签名仅可使用以下类型的参数：(a) 自身即为镜像类型或互操作类，或 (b) 与Objective-C基本类型完全对应。详见步骤3中的类型映射表及第 章《Objective-C到仓颉映射》。

Objective-C 到仓颉映射

当前版本的镜像生成器执行以下Objective-C→仓颉转换规则。

通用说明

Objective-C镜像生成器依赖Clang解析Objective-C源代码，调用时需添加-fobjc-arc选项。

标记为unavailable属性的声明将被忽略。全局声明、文件级函数及变量声明同样

会被忽略。

输出仓颉源代码中声明的顺序与输入Objective-C源代码中相应定义的顺序一致，嵌套类型定义除外。

名称

原始 Objective-C 标识符将被保留，但存在以下例外：

- 与仓颉关键字冲突的 Objective-C 标识符（如 catch、false 或 UInt32）将用反引号 `` 括起来。
- Objective-C 类与协议之间的名称冲突通过在协议名称后添加后缀 Protocol 来解决。
- 实例方法与静态方法之间的命名冲突，在可能的情况下通过在冲突发生处添加Instance或Static后缀来解决，该后缀添加于最高层级（最接近根节点）的类/协议。若冲突的实例/静态方法首次出现在同一类/协议中，则重命名静态方法。例如：

```
@interface     A
+(void)foo; @end

@interface B : A
-(void)foo;
+(void)bar;
-(void)bar; @end
```

将转换为 @ObjCMirror

```
public open class A <: ObjCId { public static
    func foo()
}

@ObjCMirror
public open class B <: A {
    public open func fooInstance()public
    static func barStatic()public open func
    bar()
}
```

- 名称不同但参数数量和类型相同的初始化方法之间的冲突无法解决。详情请参阅类

  。

类型别名

C 语言的 typedef 声明将转换为公共的仓颉类型别名。

基本类型

Objective-C 基本类型映射至对应的仓颉值类型。具有平台特定大小的 C 类型将根据其在主机平台（即镜像生成器运行平台）上的实际大小映射至仓颉类型。例如在 macOS 上可能如下所示：

Objective-C	仓颉
void	Unit
BOOL	Bool
有符号字符	Int8
短整型	Int16
int	Int32
long	Int64
long long	Int64
unsigned char	UInt8
unsigned short	UInt16
unsigned int	UInt32
无符号长整型	UInt64
无符号长整型	UInt64 浮点型
	浮点32位
double	浮点64

结构体

C 结构体如果与 CType 兼容（即只包含与 CType 兼容的类型字段），则转换为标记为 @C 的公共仓颉结构体。其他情况（例如原始 Objective-C 结构体包含指向对象的指针）目前尚不支持。

嵌套结构转换为顶级仓颉结构。

不完整的结构声明将转换为空结构（不含任何成员）。

Objective-C 结构体的字段将转换为对应 Cangjie 结构体的公共成员变量，静态修饰符得以保留。

Cangjie语言不支持位域，因此会忽略其宽度并发出警告。

与Objective-C不同，仓颉语言要求结构体所有成员变量（包括@C声明的）必须初始化。因此镜像结构体中所有字段均包含显式零初始化器。例如：

```
struct A {
    int x;
    double y;
    BOOL z;
    struct A *w;
};
```

映射到 @C

```
public struct A {
    public var x: Int32 = 0public var y:
    Float64 = 0.0public var z: Bool =
    false
    public var w: CPointer<A> = CPointer<A>()
}
```

枚举

命名C枚举声明将转换为抽象密封的仓颉类（实质上是命名空间）。枚举项将转换为使用其值初始化的公共静态常量。其类型对应于枚举显式指定的基础类型（如有），否则为 Int32。

联合体

Cangjie语言不支持C语言的联合体类型。它们会被转换为具有顺序字段的 结构体，并发出警告。

类与协议

Objective-C类与协议分别映射为仓颉类和接口。所有此类映射对象均显式实现内置的ObjCId映射接口（详见内置类型）。

在具有可变参数数量的方法中，省略号（, ...）将被忽略。

以冒号（:）标记的完整Objective-C方法名（选择器）不构成有效的仓颉标识符。此类名称将按以下规则进行混淆处理：

- 若冒号后紧跟字母，则该字母首字母大写。
- 移除所有冒号字符。

原始选择器通过仓颉侧的@ForeignName属性进行标识。

示例：

```
@interface    A
- (void)foo;
- (void)foo:(int)i;
- (void)foo:(int)i bar:(int) j;
- (void)foo:(int)i bar:(int) j baz:(int) k; @end
```

映射到 @ObjCMirror

```
public open class A {
        public open func foo(): Unit
        @ForeignName["foo:"] public open func foo(i: Int32): Unit@ForeignName["foo:bar:"] public open
        func fooBar(i: Int32, j: Int32): Unit
        @ForeignName["foo:bar:baz:"] public open func fooBarBaz(i: Int32, j: Int32, k: Int32):
}
```

在Cangjie中，覆盖其他属性的属性必须具有相同类型。此外，Cangjie属性的获取器/设置器函数名称不可随意命名。因此，Objective-C类中声明的属性仅*在可能时*才会映射为Cangjie属性。其余情况下，其获取器/设置器方法将作为普通实例成员函数进行映射。这包括在@protocol指令内声明的属性。

实例变量将映射为实例成员变量。

instancetype 被转换为当前声明的名称。

类

Objective-C @interface 类声明被转换为标记为 @ObjCMirror public open 的仓颉类。

Objective-C @interface 类别的声明与相应的仓颉类声明合并。

Objective-C @implementation 声明将被忽略。

前向类声明（@class 指令）被映射为空类（无任何成员）。

根据 Clang 文档中关于 Objective-C ARC 的方法家族部分所述，被识别为 init 方法的方法将映射为 Cangjie 构造函数，但存在两项限制：

- 若某类中两个或多个初始化方法仅名称不同（即参数数量和类型相同），镜像生成器将注释掉对应的苍楞构造函数声明并发出冲突警告。
- init方法与其他方法一样可被继承，而仓颉中的构造函数不可继承。因此，在镜像类或互操作类实例化时，映射自父类的init方法对应的构造函数不可用。

Objective-C 类的其他方法会被镜像为对应 Cangjie 镜像类型的公共开放成员函数。类方法（以“+”为前缀）的镜像会添加 static 修饰符。

协议

Objective-C 的 @protocol 指令会被镜像为标记为 @ObjCMirror public 的 Cangjie 接口。但 cjc 编译器目前尚不支持此类接口。

向前协议声明映射为空接口（不含任何成员）。

Objective-C 协议的方法映射为对应仓颉接口的公共开放成员函数，静态修饰符得以保留。

忽略 @optional 指令。

指针

修饰符 const、volatile 和 restrict 将被忽略。

指向C基本类型的指针及其类型别名将转换为CPointer，并带有相应的类型参数。

指向 C 枚举的指针将转换为指向其底层类型的 CPointer，实际枚举名称通过注释指定。

结构体指针

指向 C 结构 T 的指针转换为 CPointer<T'>，其中 'T' 是 T 的镜像类型（若该结构兼容 CType）。否则转换为 ObjCPointer<T'>（暂定名），该类型应在互操作库中实现。

函数指针

指向C函数的指针会被转换为CFunc（若函数参数类型和返回值均与CType兼容），否则则转换为ObjCFunc<F>（暂定名）。后者是互操作库中实现的类型。

块指针

指向Objective-C代码块的指针转换为ObjCBlock<F>（暂定名），该类型由互操作库实现。ObjCBlock<F>的类型参数F必须是对应的Cangjie函数类型。

类实例指针

Objective-C 类实例指针映射规则如下：

- 类实例指针（SomeClass*）映射至对应镜像类。
- Objective-C类型id通过单一协议进行收窄（例如id<NSCopying>）时，其镜像为该协议的镜像接口。
- id 通过多个协议进行收窄（例如 id<NSCopying, NSSecureCoding>）时，映射为纯粹的 ObjCId（所有 @ObjCMirror 类和接口分别实现并继承的接口），实际协议列表通过注释指定。
- 若泛型模板内部使用了泛型类型参数且指定了收窄协议，则该参数会被转换为指向该协议的引用，其名称为 ，类型参数的名称则以注释形式标注。

泛型

参数化 Objective-C 类将映射为常规的非泛型 Cangjie 类声明。原始轻量级泛型语法（如“<T>”或“<Foo>”）保留在注释中；类型参数的使用将替换为 ObjCId。

示例：

```
@interface G<T> : NSObject
- (void)f:(T)t;
@end
```

映射为 @ObjCMirror
```
public open class G/*<T>*/ <: NSObject {
    @ForeignName["f:"] public open func f(t: ?ObjCId /*T*/): Unit
}
```

然而，在当前版本中类型约束会丢失，即：

```
@interface G<T: SomeType*> : NSObject
- (void)f:(T)t;
@end
```

现在完全与上文第一个示例对应。

内置类型

镜像生成器假定以下用户可用的仓颉类型已在互操作库中实现。但当前版本尚未实现这些类型，且其名称可能变更。

Objective-C	仓颉 (*)	描述
id	ObjCId	所有 @ObjCMirror 类和接口都应实现的接口。绑定到 Objective-C id。
SEL	SEL?	绑定到 Objective-C SEL 的类。
Class	Class?	绑定到 Objective-C 类的类。
协议	协议?	绑定到 Objective-C 协议的类。
指针类型	ObjCPointer<T>?	绑定到 Objective-C 指针的结构体：参数个数大于一个时，以及当 T 不是 CType 兼容结构体时。
非C函数类型	ObjCFunc<F>?	实现CFunc的类，用于 F 为仓颉函数类型。
块类型	ObjCBlock<F>?	实现Objective-C代码块的结构体块。F 是 Cangjie 函数类型。
	__builtin_va_lisCPointer<Unit>	的辅助类型别名，当前生成器实现中技术上需要。未来可能且应当移除。

(*) 仓颉类型名称为临时命名。

(尚未)实现的功能

仓颉 SDK 的 Objective-C 互操作性支持仍在开发中。部分功能尚未实现，其他功能可能在首个正式版本中变更，另有部分功能因两种语言原理性差异无法（完全）实现。

- C语言中改变数据默认大小、打包、填充和/或对齐的特性高度依赖ABI。仓颉语言不支持此类底层特性，尤其是位域。因此镜像生成器会忽略位域宽度指定符并发出警告。
- 仓颉语言不支持C语言的联合体类型，故将其镜像为结构体并发出警告。
- 结构体中包含非CType兼容类型的字段将不被支持。
- 匿名C枚举声明将被忽略。命名枚举将映射为抽象密封的仓颉类。详情参见枚举说明。
- 在具有可变参数数量的方法中，... 将被忽略。
- 忽略 @optional 指令。
- 属性在可能时映射为属性，否则将其获取器和设置器方法映射为常规实例方法。详见类与协议章节。
- 修饰符 const、 volatile 和 restrict 将被忽略。
- init 方法映射为仓颉构造函数，此机制存在两项限制：
  - 仓颉类的两个构造函数不能具有相同数量和类型的参数，而init方法可以因名称不同而存在差异，
  - 仓颉构造函数不支持继承。详见《类》章节。
- 参数化Objective-C类将映射为常规的非泛型仓颉类声明。详情请参阅泛型部分。
- 目前尚未完全支持将@protocol声明映射至仓颉接口。
- 不支持指向参数/返回类型与 CType 不兼容的函数的指针。
- Objective-C 块不被支持。

- 将 Objective-C 类型 NSString 的值转换为 Cangjie 类型 String 的值，或将 Cangjie 类型 String 的值转换为 NSString 的值的内置辅助函数尚未实现 。

空值处理

仓颉不支持空引用概念，因此没有与Objective-C的nil值对应的机制。若将Objective-C类和协议的指针映射到仓颉类和接口，任何从Objective-C传递至仓颉的此类值都可能导致段错误。反之，仓颉也无法向Objective-C传递nil值。

因此，Option<T>枚举通常用于表示这些Objective-C类型的值，其中None代表空值，Some(r)代表非空引用值r。当T是镜像类型或互操作类时，cjc编译器会将Option<T>识别为与Objective-C兼容的类型，并相应地对T的值进行包装/拆包操作。

例如以下 Objective-C @interface 指令：

```
@interface MyContainer: NSObject

- - -
- (void)addItem:(MyItem *)item withUuid:(NSString *)uuid;
- (MyItem *)itemWithUuid:(NSString *)uuid;
- (NSString *)uuidForItem:(MyItem *)item;@property (copy)
NSArray<MyItem *> *allItems;@end
```

将映射为（为简洁起见省略注释）：

```
@ObjCMirror
open class MyContainer <: NSObject {
        - - -
    public open func addItemWithUuid(item: ?MyItem, uuid: ?NSString): Unitpublic open func
    itemWithUuid(uuid: ?NSString): ?MyItem
    public open func uuidForItem:(item: ?MyItem): ?NSString public mut prop
    allItems ?NSArray/*<MyItem>*/
}
```

Option<T> 的包装确保了即使 Objective-C 端有 nil 值渗入仓颉世界，代码也不会崩溃，但这会以性能和内存占用为代价。此方案的另一缺点是丢失了变体性。不过，支持空值标注能显著减轻引用包装的影响——至少在仓颉代码中调用 iOS API 时如此。

注意：对于映射到常规 CPointer<T> 类型的低级 C 类型而言，此问题不存在，因为后者提供了显式的空值检查

函数。

变体丢失

Objective-C 镜像类型和互操作类采用 Option<T> 包装时存在一个限制：此类包装类型在其他所有方面均遵循 Cangjie 的语义。具体而言，Option<T> *对类型参数 T 具有不变性*：若 U 是 T 的子类型，则 Option<U> 并非 Option<T> 的子类型，除非 T 与 U 属于相同类型。 对于镜像类型而言，这意味着任何依赖返回类型协变性的重写Objective-C方法，在使用Option<T>封装时可能无法实现同等镜像效果。此类方法的镜像返回值类型必须从被重写的方法中继承而来。

示例：

假设Objective-C类Foo是类Bar的直接父类：

```
@interface Foo : NSObject
@end

@interface Bar : Foo
@end
```

而类 C 声明了一个返回 Foo 实例的方法 get：

```
@interface C : NSObject
- (Foo*) get;
@end
```

C的子类可重写get方法，使用更精确的返回类型Bar：

```
@interface D : C
- (Bar*) get;
@end
```

若不使用Option<T>包装，所有这些类都可以映射为：

```
@ObjCMirror
open class Foo <: NSObject {}

@ObjCMirror
open class Bar <: Foo {}

@ObjCMirror
open class C <: NSObject {open func
    get(): Foo
}

@ObjCMirror
open class D <: C {
```

```
    open func get(): Bar                    // 返回类型协变性在实际应用中
}
```

但若 get() 可能返回 nil，应用程序崩溃将不可避免。

使用Option<T>进行包装可以确保安全性，但所有重写方法的返回类型都必须降级为原始方法的返回类型：

```
@ObjCMirror
open class Foo <: NSObject {}

@ObjCMirror
open class Bar <: Foo {}

@ObjCMirror
open class C <: NSObject {
    open func get(): Option<Foo>
}

@ObjCMirror
open class D <: C {
    // open func get(): Option<Bar>          // 错误，`Option<T>` 不是 T 的协变类型open func
    get(): Option<Foo>                       // 允许，但返回类型会被降低
}
```

空值标注可部分缓解此问题。

空值标注

空值标注关键词随 XCode 6.3 版本引入 Objective-C，旨在与新型 iOS/OS X 开发语言 Swift 实现更佳兼容。当时所有 iOS API 定义均增强了空值标注，以降低 Swift 代码使用可选类型的频率。

6.3版本中引入，旨在更好地与新型iOS/OS X开发语言Swift集成。所有iOS API的定义均增强了空值标注功能，以降低使用这些API的Swift代码中可选类型的使用频率。

Objective-C 空值标注

关键字 nullable 和 nonnull 可用于 Objective-C 属性、方法返回类型及方法参数类型。它们分别表示该实体可能持有或接受 nil 值，也可能不持有或接受 nil 值。而 (极其罕见的) null_unspecified 注解则表示该实体的值是否为 nil 无法确定。

此外，任何指针类型均可标注为__ nullable、__nonnull 或__ null_unspecified 进行注释，其语义相同。

最后，属性还可标注 null_resettable，这意味着其获取器永远不会返回 nil，但向设置器传递 nil 时会将属性重置为某个默认值。

详情请参阅《Objective-C API 中空值可选性的标注规范》

因此，所有标注为非空的Objective-C引用类型均无需进行Option<T>封装。换言之，Objective-C镜像生成器仅对原始代码中未标注nonull或_Nonnull的属性、方法返回类型及方法参数类型生成封装类型。

假设本节开头示例添加了空值标注如下：

```
@interface MyContainer: NSObject
- - -
- (void)addItem:(nonnull MyItem *)item withUuid:(nonnull NSString *)uuid;
- (可空 MyItem *)itemWithUuid:(非空 NSString *)uuid;
- (nullable NSString *)uuidForItem:(nonnull MyItem *)item; @property (copy, nonnull) NSArray<MyItem *> *allItems; @end
```

镜像生成器将跳过非空实体的Option<T>封装（为简洁起见省略@ForeignName注释）：

```
@ObjCMirror
open class MyContainer <: NSObject {
    - - -
    public open func addItemWithUuid(item: MyItem, uuid: NSString): Unitpublic open func itemWithUuid(uuid: NSString): ?MyItem
    public open func uuidForItem:(item: MyItem): ?NSString public open prop allItems: NSArray
}
```

注意：目前无法在传播该注解语义的同时将 null_resettable 属性映射到仓颉属性中，因此后者将被完全视为可空属性。

若您希望从Cangjie访问的Objective-C代码未充分利用空值标注功能，建议在使用本文所述Cangjie SDK互操作特性前，至少为相关属性添加非空标注。此举可显著减少对应镜像 类型中Option<T>封装的使用，使互操作类代码更简洁易读。

Objective-C镜像生成器参考

使用前提

使用镜像生成器前，请务必运行Cangjie SDK中的envsetup.sh脚本。

您需要了解框架和库中所有头文件的路径名称，这些框架和库定义了您即将镜像的类的所有依赖项。这包括iOS标准库头文件的位置，以及Objective-C编译器在构建项目时通常会查找头文件的所有目录。

命令行语法

ObjCInteropGen [-v] [--*mode=normal*config-file*]

-v

生成详细输出。

--mode=normal

强制使用正常操作模式。其他模式仅用于镜像生成器本身的开发和测试。当前版本中，若指定*配置文件，则必须*使用 --mode=normal 选项，但未来版本中该选项将可选。

config-file

配置文件的路径名。

配置文件语法

Objective-C镜像生成器的配置文件采用TOML语法的纯文本文件格式。该文件指定：

- 输出目录
- 源Objective-C头文件名（.h文件）
- 输出仓颉包的名称及其镜像类型的分布
- 需要特殊处理的类型映射

作为正则表达式解释的字符串值必须遵循EC- MAScript正则表达式语法。

输出目录根目录

[输出根目录]表中的每个条目都定义了本地文件系统中目录路径名的符号名称。镜像生成器将使用该路径名作为一个或多个特定包输出目录的公共根目录，这些目录通过相应[[packages]]数组元素的output-root属性设置。

示例：

```
[output-roots.lib]
路径 = "./lib/src"
```

```
[输出根目录.app]
路径 = "./main/src"

[[包]]
package-name = "com.vendor1.lib1"
输出根目录 = "lib"          # 输出路径为 "./lib/src/com/vendor1/lib1"
过滤器 = ...

[[包]]
package-name = "com.vendor2.lib2"
输出根目录 = "lib"          # 输出至 "./lib/src/com/vendor2/lib2"
过滤器 = ...

[[包]]
package-name = "com.mycompany.app"
输出根目录 = "app"          # 输出至 "./main/src/com/mycompany/app"
过滤器 = ...
```

源文件

[sources] 表中的每个条目都指定了一组单独的头文件，镜像生成器必须将其作为输入。

属性：

路径 (必填)

包含镜像生成器必须读取的各个头文件路径名的字符串数组。

参数（可选）

字符串数组，包含镜像生成器在处理路径中列出的源文件时传递的Clang选项。

示例：

```
[sources.all]
paths = ["original-objc/M.h"]
```

附加 Clang 参数

[sources-mixins] 表中的每个条目都指定了一个正则表达式，用于匹配 [sources] 表中的一个或多个键，以及在处理其对应条目中的头文件时必须传递给 Clang 的附加参数。

属性：

sources（必填）

包含正则表达式的字符串。

arguments-prepend arguments-
append（可选）

字符串数组，镜像生成器将在处理任何[sources]表条目中与sources属性指定的正则表达式匹配的源文件时，将这些参数作为选项传递给Clang。

arguments-prepend 和 arguments-append 中列出的选项将分别置于每个匹配 [sources] 条目中 arguments 属性指定的选项之前和之后（若存在）。

示例：

```
[sources.UIWidgets]
paths = ["objc/UIWidgets.h"]
arguments = [ "-I", "/usr/local/include/share/Widgets" ]

[sources.UIPanels]
路径 = ["objc/UIPanels.h"]
参数 = [ "-I", "/usr/local/include/share/Panels" ]

[sources-mixins.UI]
# 为 UIWidgets 和 UIPanels 添加以下 Clang 参数
sources =
["UI.+"]arguments-append
= [
    "-I", "/usr/local/include/Frameworks/AcmeUI"
]
```

包

[[packages]]数组中的每个条目指定一个目标仓颉包名称、一组定义哪些Objective-C实体将映射到该包的名称过滤器，以及可选的该包专属输出目录。

属性：

package-name（必填）

包含目标仓颉包名称的字符串。

输出路径 (可选)

字符串，包含镜像生成器应将指定包的输出文件放置到的目录*精确*路径名。若路径中存在不存在的目录，镜像生成器将尝试创建它们。

输出根目录 (可选)

包含输出根目录表中键值的字符串。目标仓库包的名称将附加到对应输出根目录的值后。

表项中的点号 . 将被文件分隔符 / 替换，所得路径名将被视为 output-path 的值使用。

若未设置输出根目录，且[输出根目录]仅含一个元素，则采用该条目的路径名。否则将显示错误。

示例：

```
[输出根目录.主]
path="./cj-mirrors"

[[packages]]
package-name = "objc.foundation" output-root
= "main"
```

输出文件将存放于 ./cj-mirrors/objc/foundation 目录下。过滤器（必填）

定义名称过滤器集的表格，用于从源文件中筛选出必须映射至指定仓颉包的Objective-C实体声明。详情参见名称过滤器说明。

示例：

```
# Foundation框架
[[packages]]
package-name = "objc.foundation" filters =
{ include = "NS.+" }
```

名称**过滤器**　每个名称过滤器本身是一个TOML表格，包含：

- 仅包含以下属性之一：include（包含）、exclude（排除）、union（并集）、intersect（交集）和not（非），以及
- 可选地包含 filter *属性及/或* filter-not 属性。各属性说明如下：

include

值可为正则表达式或其数组。若值为单个正则表达式，仅匹配该表达式的名称通过过滤；若值为数组，名称只需匹配数组中任意单个正则表达式即可通过。

示例：

```
# 仅包含名称以"NS"开头的实体：
filters = { include = "NS.+" }

# 仅包含名称以"Foo"开头或以"Bar"结尾的实体：
filters = { include = ["Foo.*", ".*Bar"] }
```

exclude

与include相反（参见上文）。通过include过滤器（使用给定值）的名称无法通过排除过滤器（使用相同值），反之亦然。

示例：

```
# 包含除名称以"INTERNAL_"开头的实体外的所有实体：
filters = { exclude = "INTERNAL_.+" }并集
```

一个组合两个或多个过滤器的运算符。其值必须是过滤器数组。通过其中任意单个过滤器的名称，即视为通过整个联合过滤器。

示例：

```
# 等同于上述第二个 `include` 示例：
filters = { union = [ { include = "Foo.*" }, { include = ".*Bar" } ] }intersect
```

组合两个或多个过滤器的运算符。其值必须是过滤器数组。名称必须通过所有过滤器才能通过整个交集过滤器。

示例：

```
// 添加否定过滤器：
filters = {    intersect = [ {       include = "NS.+" },
                                    { 排除 = "NSAccidentalClash" } ] }
```

not

一个反转过滤器含义的运算符。其值必须是单个过滤器：

示例：

```
// 添加否定过滤器的另一种方式：
filters = { intersect = [ { include = "NS.+" },
                                    { not = { include = "NSAccidentalClash" } } ] }
```

filter
filter-not （可选）

这些属性必须与其他属性混合使用，即它们不能是过滤器表中的唯一属性。其值可以是正则表达式或正则表达式数组，与include和exclude属性的值相同。

filter 属性将通过主过滤器筛选后的集合进一步缩减，仅保留匹配指定单个正则表达式或数组中任意正则表达式的名称。

filter-not 将通过主过滤器的集合进一步缩减，仅保留不匹配给定单个正则表达式或数组中任何正则表达式的名称。

filter 和 filter-not 实际上分别是包含过滤器和排除过滤器的交集操作的简写形式。它们可以同时指定。

示例：

```
# 未使用 filter-not 时：
filters = { intersect = [ { include = "NS.+" },
                                    { exclude = "NSAccidentalClash" } ] }
# 使用 filter-not：
filters = { include = "NS.+", filter-not = "NSAccidentalClash" }

# 'filter' 和 'filter-not' 可同时使用：
filters = { include              = ".*Fizz.+",
            filter               = ".+Buzz.*",filter-not
            = ".*FizzBuzz.*" }
```

类型替换

[[mappings]] 是一个表数组，每个表都为某种类型的替换列表添加另一种类型的替换项。几乎所有类型的引用（除 int 等 C 基本类型外）都可以替换为另一种类型的引用。

示例：

```
# 将类型 id （Objective-C 类型层次结构的根节点） # 在所有位置替换为 NSObjectProtocol。
[[mappings]]
id = "NSObjectProtocol"
```

配置文件导入

imports 是一个字符串数组，每个元素包含另一个配置文件的路径名，其设置将被添加到当前配置中。导入文件中 packages 和 mappings 数组的条目将追加到导入文件中已存在的条目之后。

支持嵌套导入，循环导入将被检测并报错。

示例：

```
import = "../common.toml"
```